שם ביה"ס : **קרית נוער**

סמל ביה"ס : **140129**

קרית נוער ירושלים

עבודת גמר

למילוי חלקי של הדרישות לקבלת תואר

הנדסאי תוכנה

משחק רשת דמקה

מגיש

אמיתי אגאי 324852292

בהנחיית מר אילן פרץ

שנה"ל תשפ"ב

2022

תוכן עניינים

הצעת הפרויקט 4
1. הצהרת הסטודנט ואישור הגשה 5
2. מבוא 6
3. מדריך למשתמש 7
3.1 תיאור המשחק 7
3.2 דרישות טכניות 8
3.3 הרצת המשחק והנחיות שימוש 8
3.3.1 הרצת השרת והשימוש בו 9
3.3.2 הרצת הלקוח והשימוש בו 12
3.3.3 הפעלת המשחק וממשק משתמש גרפי (GUI) 15
4. מדריך למתכנת 20
4.1 הדרישות ממערכת התוכנה 20
4.2 ארכיטקטורת המערכת 21
4.2.1 מודל שרת-לקוח Client-Server Model 21
4.2.2 פרוטקול תקשורת TCP/IP 22
4.2.3 שקעים Sockets 22
4.2.4 מסד הנתונים 22
4.2.5 תהליכונים Threads 25
4.3 תהליכים עיקריים וזרימת המידע 25
4.3.1 חיבור לקוחות והתחלת סשן משחק 25
4.3.2 התחברות לקוח 26
4.3.3 סשן משחק בין 2 שחקנים 27
4.3.4 בקשת סטטיסטיקה מהשרת 29
4.4 מחלקות הפרויקט 30
4.4.1 תרשים UML של המחלקות בצד השרת 30
4.4.2 תיעוד המחלקות בצד השרת 31
4.4.3 תרשים UML של המחלקות בצד הלקוח 42
4.4.4 תיעוד המחלקות בצד הלקוח 43
4.4.5 תיעוד המחלקות המשותפות ללקוח ולשרת 47
4.5 אלגוריתמים ופעולות נבחרות 58
4.5.1 אלגוריתם מינימקס עם גיזום 58
4.5.2 פונקציית הערכה 61
4.5.3 הרצת משחק- runGame() 64
4.5.4 proccessMsgFromServer 66
4.5.5 getAllPossibleMoves 69

4.6 מבני נתונים עיקריים 70
5. סיכום אישי 71
5.1 אתגרים וקשיים איתם התמודדתי 72
5.2 מה הפרויקט תרם לי 72
5.3 הצעות לשיפור 72
5.4 מסקנות 73
6. תודות 73
7. ביבליוגרפיה 73

הצעת הפרויקט

סמל מוסד: **140129**
שם מכללה: **קריית נוער – כנפי רוח, ירושלים.**
שם הסטודנט: **אמיתי אגאי**
ת"ז הסטודנט: **324852292**

שם הפרויקט: **משחק דמקה**
תיאור הפרויקט: **משחק דמקה שמתנהל בין שני שחקנים ברשת המקומית**

הגדרת הבעיה האלגוריתמית: **הבעיה הראשית שהפרויקט שלי פותר היא משחק דמקה מרובה שחקנים (שחקן רשום/ אורח/ AI) כולל סטטיסטיקות, שמירת תוצאות משחקים ושמירת משחקים שלא נגמרו במסד נתונים (רק עבור משתמשים רשומים במשחק נגד AI) שניתן יהיה להמשיך אותם לאחר מכן**

רקע תיאורטי בתחום הפרויקט: **הפרויקט מסכם את הנושאים שלמדתי לאורך שנתיים של לימודי תוכנה: רקורסיה, object oriented, ירושה ופולימורפיזם, SQL, תעבורת נתונים ברשת, MVC, תכנות מקבילי ומבוזר, עיצוב ממשק משתמש גרפי**

תהליכים עיקריים בפרויקט: **התחברות לשרת (כשחקן רשום/ אורח) ובחירה מול מי לשחק (ממוחשב/ אנושי), השרת עובד במקביל לצורך שירות של הרבה משתמשים שמתחברים אליו, יצירת משחק על ידי שידוך 2 שחקנים אנושיים/ משחק מול AI, שמירת נתונים במסד הנתונים (שמירת נתונים על משחקים שהסתיימו, שמירת משחקים שלא הסתיימו עבור המשך בעתיד), הפקת סטטיסטיקות ממסד הנתונים ועדכון שלו**

תיאור הטכנולוגיה: **מודל שרת/ לקוח, שימוש בסוקטים של TCP/IP, שימוש ב-threads**

שפת התכנות צד לקוח ושרת **שפת JAVA**

פרוטוקולי תקשורת: **פרוטוקול TCP/IP**

תיאור מסד נתונים: **מסד נתונים של access**

לוחות זמנים :

	אוקטובר	נובמבר	דצמבר	ינואר	פברואר	מרץ
ניתוח המערכת						
תכנון מבני הנתונים ואלגוריתמים						
כתיבת הקוד						
עיצוב ממשק המשתמש						
ניפוי שגיאות ובדיקות איכות						
הרצת התוכנית						

1. הצהרת הסטודנט ואישור הגשה

הצהרת הסטודנט

שם הסטודנט: אמיתי אגאי ת"ז: 324852292

אני החתום מטה, מצהיר בזאת כי פרויקט הגמר וספר הפרויקט המצ"ב נעשו על ידי בלבד.

פרויקט הגמר נעשה על סמך הנושאים שלמדתי באופן עצמאי, ועל בסיס הנחייתו של המנחה האישי. מקורות המידע בהם השתמשתי לביצוע פרויקט הגמר מפורטים ברשימת המקורות (ביבליוגרפיה) שנמצאת בסוף ספר הפרויקט.

אני מודע לאחריות שהנני מקבל על עצמי על ידי חתימתי על הצהרה זו שכל הנאמר בה אמת ורק אמת.

חתימת הסטודנט : _____ תאריך : _____

אישור המנחה האישי

הריני מאשר שהפרויקט בוצע בהנחייתי, בדקתי את קוד הפרויקט וקראתי את ספר הפרויקט ומצאתי כי הוא מוכן לצורך הגשת הסטודנט להגנה על פרויקט גמר.

שם המנחה : _____ חתימה : _____ תאריך : _____

אישור ראש המגמה

הריני מאשר הגשת הסטודנט להגנה על פרויקט הגמר.

שם ראש המגמה : _____ חתימה : _____ תאריך : _____

2. מבוא

פרויקט הגמר מממש תוכנה **למשחק רשת** מבוסס **משחק לוח לשני שחקנים** בשם "**דמקה**". הסבר על המשחק, הפעלתו והשימוש בו, נמצא במדריך למשתמש בפרק 3 הכולל הצגת תצלומי מסך והסברים מפורטים לכל השלבים.

המשחק תוכנן ופותח על פי הדרישות ממערכת התוכנה הבאות, ואשר מפורטות בפרק 4:

- ריבוי משחקים במקביל בין שחקנים אנושיים שמתחברים מרחוק באמצעות רשת האינטרנט.
  לכן מערכת התוכנה עושה שימוש בארכיטקטורה המבוססת על **מודל שרת/לקוח** (Client/Server) הכולל עבודה עם **שקעים** (Sockets) בפרוטוקול TCP/IP, ושימוש בתהליכונים **Threads**.

  בסעיף 4.2 יש הסבר מעמיק על ארכיטקטורת המערכת ומרכיביה.

- לאפשר לשחקן רשת (אנושי) לבחור לשחק מול שחקן ממוחשב עם בינה.
  לכן מערכת התוכנה עושה שימוש **באלגוריתם Minimax** למימוש השחקן הממוחשב.

  בסעיף 4.5.1, יש הסבר מעמיק על אלגוריתם מינימקס המממש את פעולת השחקן הממוחשב.

- לאפשר התחברות (Login) של שחקנים אורחים ורשומים, שמירת תוצאות משחקים שהסתיימו, שמירת משחקים שנפסקו וטעינתם בהמשך,לקבלת סטטיסטיקות ולאפשר פעולות עדכון.
  לכן מערכת התוכנה עושה שימוש **במסד נתונים** מסוג Access ובו שמורות טבלאות המכילות מידע על השחקנים הרשומים, על תוצאות המשחקים, על מצב המשחקים שלא הסתיימו ועוד...

  בסעיף 4.2.4 יש הסבר מעמיק יותר על מסד הנתונים ותיאור כל הטבלאות השמורות בו.

- לכתוב את קוד התוכנה בשפת Java תוך שימוש בעקרונות של תכנות מונחה עצמים מתקדם.
  לכן נעשה שימוש במחלקות, **ירושה ופולימורפיזים**, **ממשקים** (Interfaces), **אוספים גנריים**, וכמובן שימוש **בתהליכונים** (Threads) לצורך ניהול **משחקים במקביל**.

  בסעיף 4.4 יש הסבר מקיף על כל המחלקות בפרויקט (ממשקי API, תרשימי UML, תיעוד...).
  בסעיף 4.2.5 יש הסבר מעמיק על השימוש בתהליכונים ולמה היה צריך אותם.

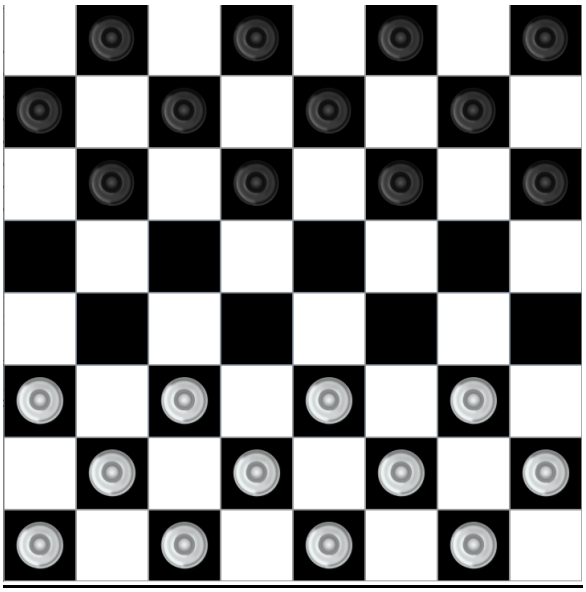
פרק 5 מסכם את החוויה האישית שלי במהלך העבודה על הפרויקט – התמודדות עם קשיים ובעיות שהיו, הכלים והמיומניות שרכשתי תוך כדי העבודה על הפרויקט ושאני לוקח איתי להמשך, המסקנות שהגיעתי אליהן לאחר סיום הפרויקט, הצעות לשיפור הפרויקט לו היה לי יותר זמן לעבוד עליו, ועוד.

3. מדריך למשתמש

בחלק זה יוסבר על המשחק, הלוח והכלים, חוקי המשחק והמהלכים. בנוסף נתאר את הקבצים המוגשים ביחד עם ספר פרויקט זה (שהם חלק בלתי נפרד מהפרויקט כולו), ונסביר איך להריץ את התוכנה ואיך להשתמש בה מנקודת מבטו של **המשתמש**.
נתחיל בפירוט הדרישות הטכניות להרצה, נמשיך בהסבר על קבצי ההרצה ואופן הרצתם, ולבסוף נסביר איך לשחק במשחק הלכה למעשה – נעשה זאת על ידי תיאור ממשק המשתמש הגרפי (GUI) בלווי תצלומי מסך להמחשה.

3.1 תיאור המשחק

משחק לוח בגודל 8X8 בין 2 שחקנים. שחקן אחד משחק עם הכלים השחורים ואחד עם הלבנים. לכל שחקן יש 12 כלים בתחילת המשחק, כך שב3 השורות העליונות על המשבצות השחורות ממוקמים 12 כלים שחורים ועל 3 השורות התחתונות על המשבצות השחורות ממוקמים 12 כלים לבנים. השחקן בעל הכלים הלבנים מתחיל ראשון לשחק. המטרה של המשחק היא לאכול את כל הכלים של היריב, כאשר הראשון שמצליח מנצח. במידה ויש לשני השחקנים אותה כמות מלכים ובמשך 5 תורות לכל אחד לא קרה כלום (כלי נאכל או מישהו ניצח) המשחק יסתיים בתיקו.
להלן לוח משחק התחלתי:



כל שחקן בתורו לוחץ על כלי שברצונו להזיז. לאחר הלחיצה יצבעו בצהוב המשבצות אותם יוכל הכלי שנבחר לבצע. כלי לבן יכול לנוע למשבצות השחורות הסמוכות באלכסון שמעליו וכלי שחור

יכול לנוע למשבצות השחורות הסמוכות באלכסון מתחתיו (כלים לבנים נעים לכיוון מעלה הלוח והשחורים לכיוון מטה). כאשר באחת המשבצות באלכסון עומד כלי יריב והמשבצת הבאה באלכסון ריקה, ניתן לעבור אליה ובכך לאכול את הכלי היריב (להוציא אותו מהמשחק). מלכים אלו כלים שהצליחו להגיע לקצה השני של הלוח (השחורים למשבצת הכי תחתונה בלוח והלבנים לעליונה ביותר) והם יכולים לנוע לאורך כל האלכסונים שלהם עד סוף הלוח או עד שיש שני כלים יריבים אחד אחרי שני באלכסון (תמונות להמחשה בחלק 3.3.3 הפעלת המשחק וממשק משתמש גרפי)

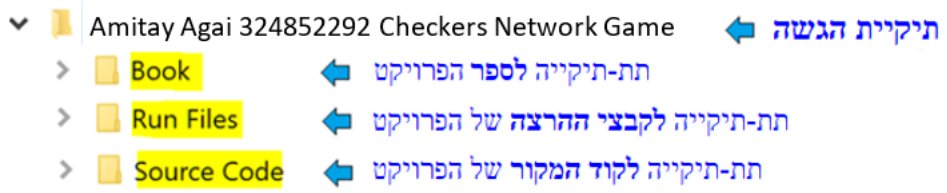
3.2 דרישות טכניות

קוד התוכנה הורץ ונבדק על מחשבים עם המאפיינים הבאים, שהם גם הדרישות הטכניות להרצת קבצי הפרויקט שעליהם :

- ✓ מחשב עם מערכת ההפעלה Windows 10 וזיכרון ראשי עם לפחות 1GB. I7
- ✓ התקנת Java Runtime Environment בגרסה 8 ומעלה.
- ✓ קישוריות לרשת ופורטים פתוחים.
- ✓ עכבר + מקלדת.

3.3 הרצת המשחק והנחיות שימוש

בתיקיית הפרוייקט המצורף יש 3 תת-תקיות :



יש להיכנס לתת-התיקייה **Run Files** שבתוכה הקבצים הבאים :

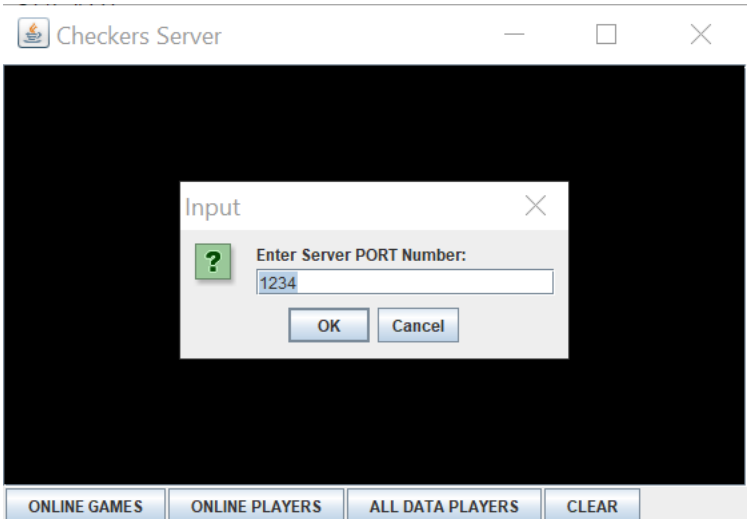
הרצת המשחק מצריכה הרצה של שני מודולים ניפרדים – מתחילים בהפעלת **השרת** ע"י הרצת הקובץ **CheckersServerProj.jar**, שמיד ממתין לחיבור לקוחות (שחקני-רשת אנושיים). הרצה זו **חד-פעמית**.

כעת מפעילים את **הלקוח** ע״י הרצת הקובץ **CheckersClientProj.jar** שמייצג שחקן-רשת יחיד. ניתן להריץ **מספר לקוחות** כדי ליצור מספר שחקנים. עבור כל שני שחקנים, השרת ״משדך״ ויוצר עבורם משחק.

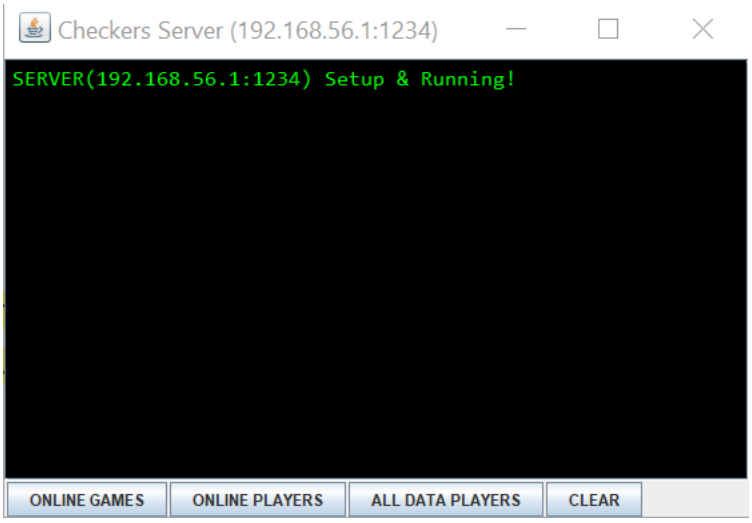
בסעיפים הבאים נסביר בצורה מפורטת, באמצעות תצלומי מסך, איך להריץ ולהשתמש בשרת ובלקוח, איך להתחיל משחק בין שני שחקנים וכמובן איך לשחק.

3.3.1 הרצת השרת והשימוש בו

על מנת להפעיל את השרת נלחץ על קובץ ה-jar. עם הלחיצה יתקבל החלון הבא:



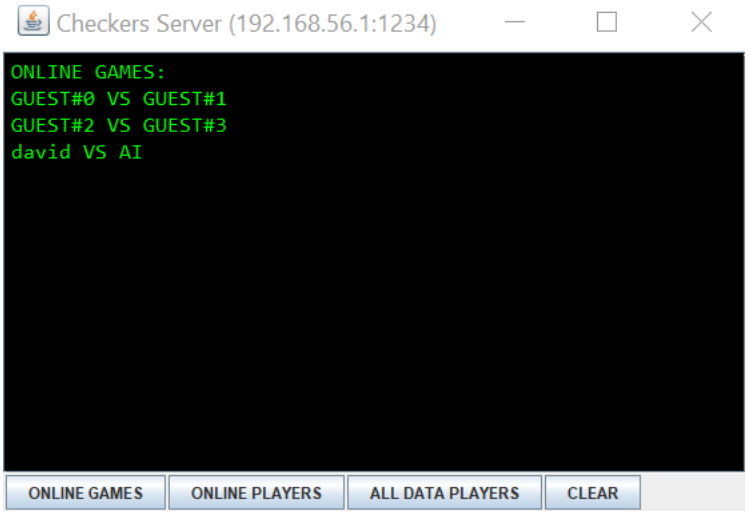
בחלון ה- input בקופץ השרת מבקש להזין את הפורט עליו הוא ירוץ (בברירת מחדל הגדרנו את הפורט כ- 1234). לאחר בחירת הפורט לוחצים על OK והשרת יתחיל לרוץ וימתין להתחברות לקוחות. לאחר מכן השרת מחכה זה המצב שיתקבל:



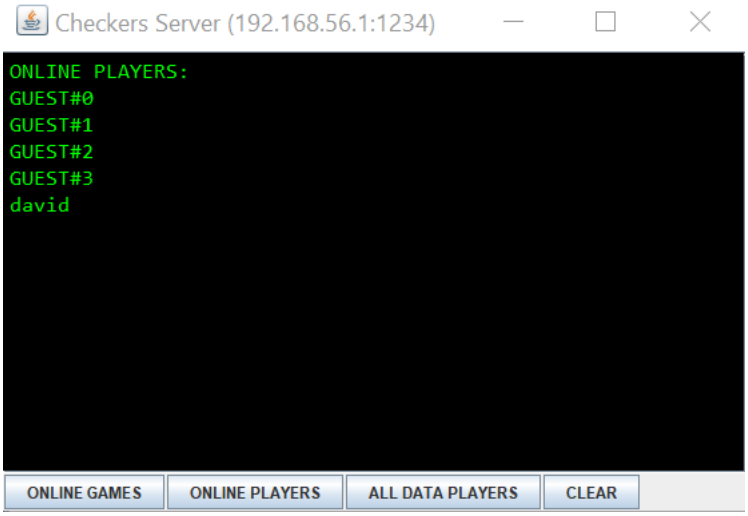
בחלון ניתן לראות:

- מסך הודעות של דברים שקורים שצד השרת (התחברות וניתוק לקוחות וכו'):
- בחלון השרת ישנם 4 כפתורים. נסביר על כל אחד מהם:

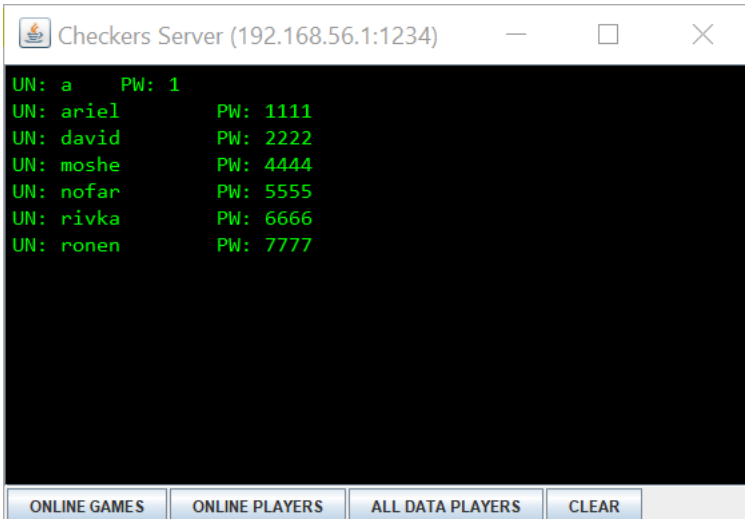
ONLINE GAMES- כפתור שמטרתו להציג את כל המשחקים שמשוחקים כרגע. לדוגמה:



ONLINE PLAYERS- בעת לחיצה על הכפתור יוצגו כל השחקנים שפעילים כעת. לדוגמה:



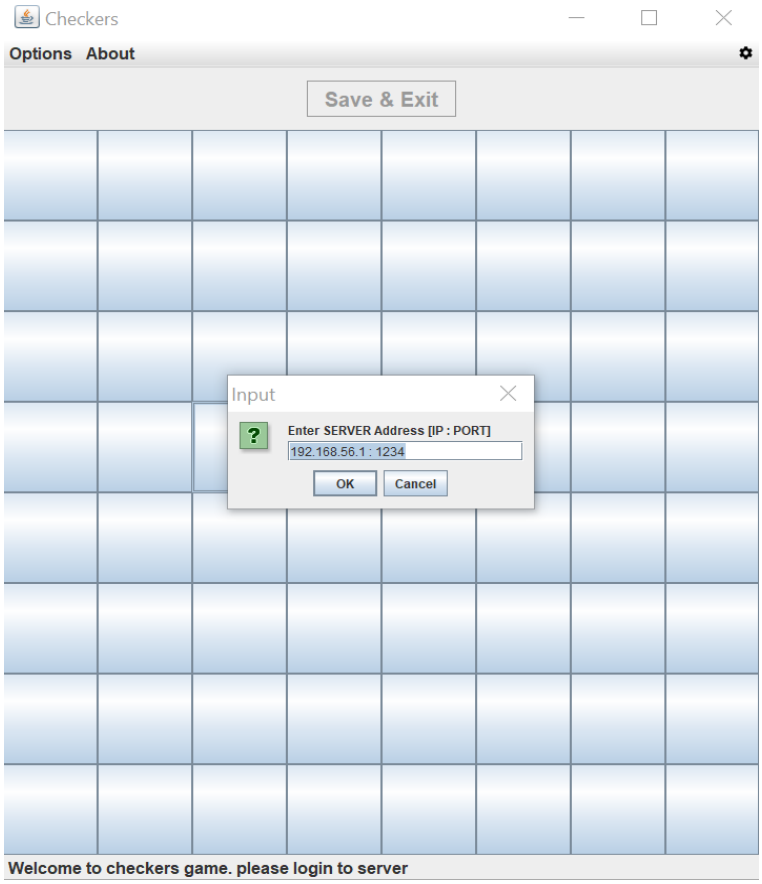
ALL DATA PLAYERS- מציג את כל השחקנים הרשומים שקיימים במסד הנתונים. לדוגמה:



CLEAR- בעת לחיצה על הכפתור כל ההודעות שהוצגו בשרת ימחקו והלוח יהיה נקי מהודעות

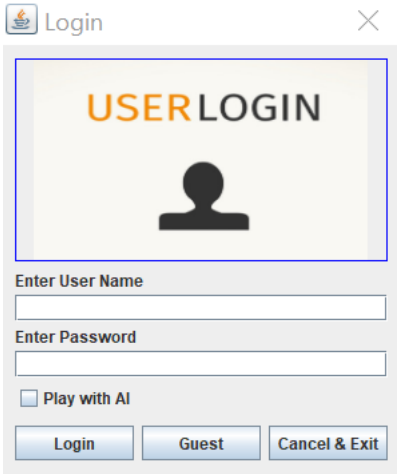
3.3.2 הרצת הלקוח והשימוש בו

לאחר שהרצנו את השרת נריץ את קובץ ה- jar של ה- client. לאחר הרצת הקובץ יתקבל המסך הבא:



בחלון ה- input שקופץ יש להכניס את ה- IP והפורט עליו נמצא השרת (מצד שמאל של הנקודותיים זה הIP ומצד ימין זה הפורט).

לאחר שהכנסנו פרטים נכונים יקפוץ החלון הבא:



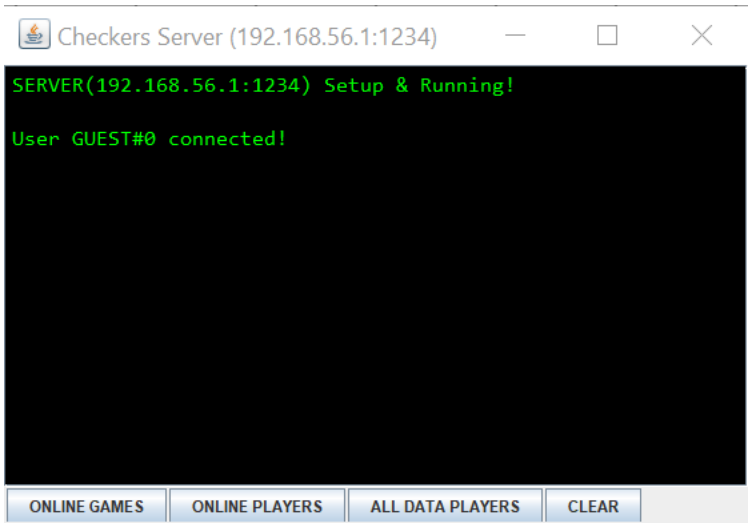
בחלון הזה מזדהים לצורך התחברות לשרת. ישנם 2 סוגי כניסות:

<u>התחברות כשחקן רשום-</u> לצורך זה יש להכניס בחלון הטקסט הראשון שם משתמש ובשני ססמא. בחלון הסימון הקטן שמתחתם- אם הוא נותר לא מסומן המשחק יתבצע נגד שחקן אנושי אחר (במידה ויש מישהו מחכה המשחק יתחיל מיד. במידה ולא יש צורך להמתין שעוד אחד יתחבר) יש לסמן אותו אם רוצים לשחק מול המחשב. לאחר שהוכנסו כל הפרטים יש ללחוץ על login.

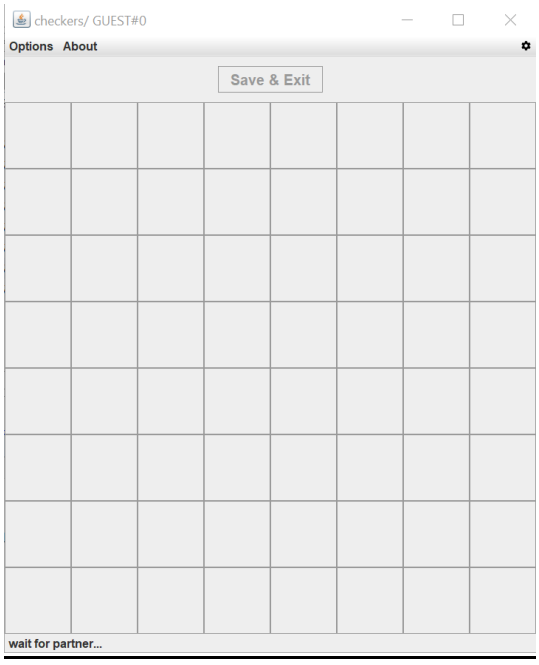
<u>התחברות כאורח-</u> במידה ואין משתמש רשום ניתן להתחבר כאורח (לשחקן אורח לא נשמרות סטטיסטיקות והוא לא יכול לקבל אותם). במידה ורוצים לשחק נגד המחשב יש לסמן את תיבת הסימון play with AI (במידה ולא תסומן המשחק יתבצע מול שחקן אנושי כלשהו).

<u>יציאה-</u> במידה והמשתמש החליט שלא להתחבר לשרת אלא להתנתק החלון יסגר.

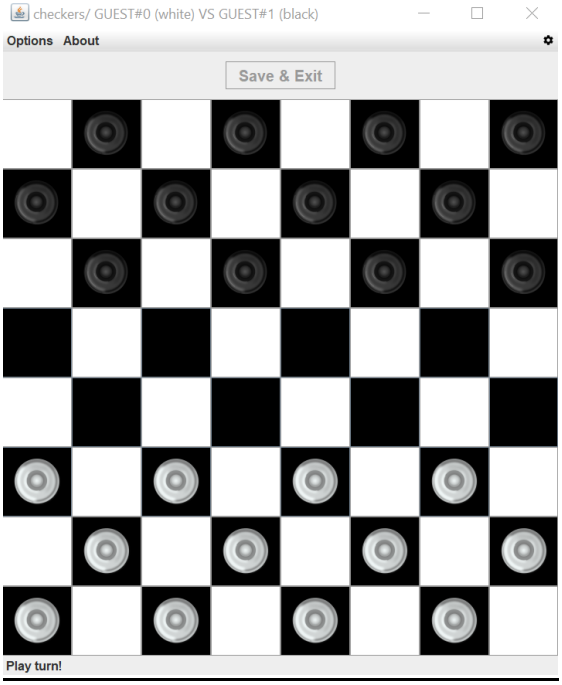
לאחר שהשחקן מתחבר מתקבל חיווי בשרת:



במידה ובחרנו באפשרות של התחברות מול שחקן אנושי אחר (לא סומנה תיבת play with AI)
ואין עדיין שחקן אנושי שמחכה, אנו נהיה בהמתנה עד שיתחבר שחקן אחר :

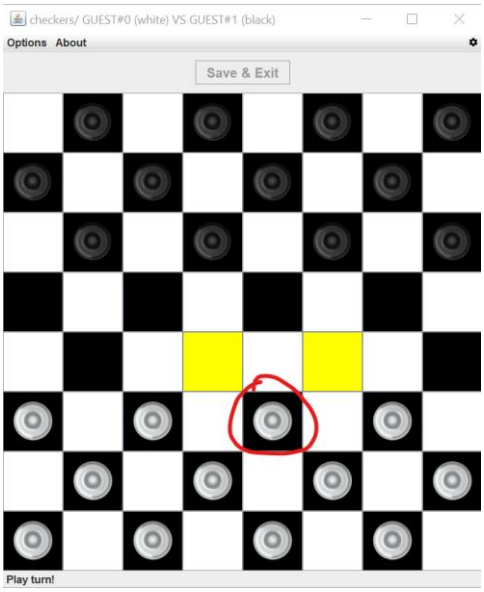


לאחר שיתחבר עוד שחקן אנושי נוסף או שהתחברנו ומישהו כבר המתין או שבחרנו באופציה של משחק נגד שחקן ממוחשב המשחק יתחיל מיד :

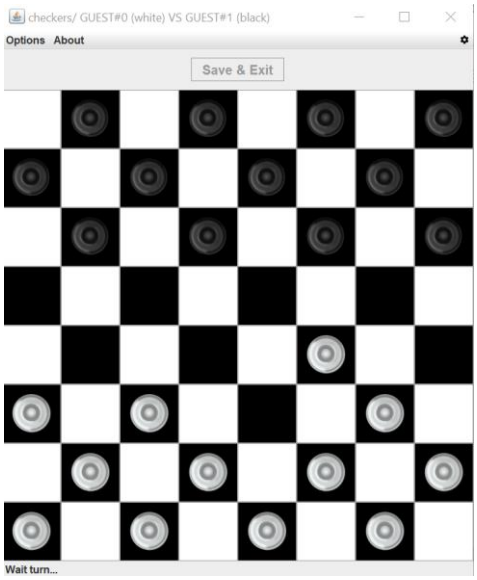


3.3.3 הפעלת המשחק וממשק משתמש גרפי (GUI)

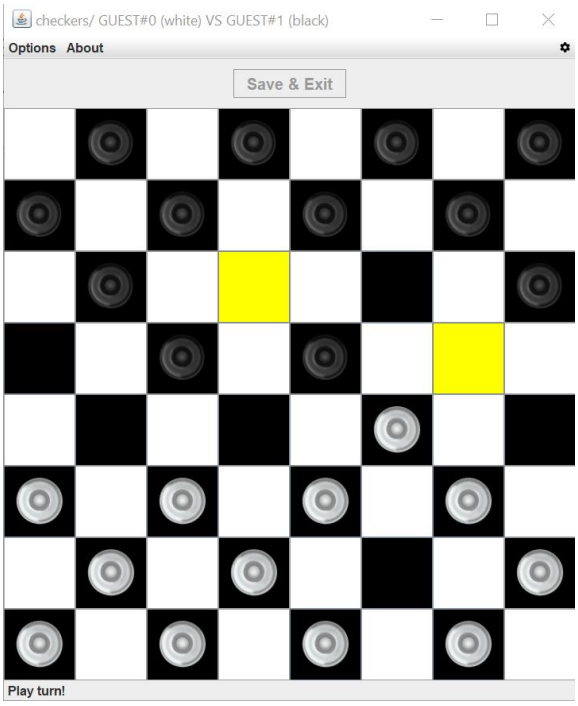
לאחר שהתחיל משחק, השחקן הראשון שהתחבר הוא יהיה בצבע לבן ויתחיל לשחק ראשון והשני שהתחבר יהיה בצבע שחור. כל כלי שניתן יהיה להזיז יהיה ניתן ללחוץ עליו וכל אחד שאין לא אפשרות תזוזה יהיה חסום ללחיצות. בעת לחיצה על כלי, יצבעו בצהוב כל המשבצות אליהן הוא יכול לעבור. אם תיבחר אפשרות חוקית (משבצת צהובה כלשהי) המהלך ייעשה. במידה ונרצה לבחור כלי אחר לתזוזה במקום הכלי שנבחר, ניתן ללחוץ שוב על הכלי שנבחר וללחוץ על כלי אחר שברצוננו להזיז במקום. לאחר ביטול בחירת הכלי, המשבצות הצבועות בצהוב יחזרו לצבעם המקורי והכלי החדש שיבחר ייצבעו אפשרויות התזוזה שלו. בדוגמה הבאה, לאחר שלחצתי על הכלי המוקף בעיגול נצבעו המשבצות האפשריות אליו יוכל להתקדם:



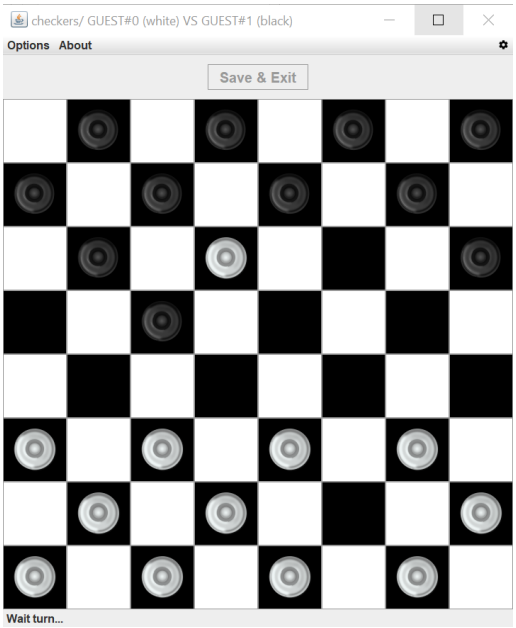
לאחר שבחרתי במשבצת הצהובה הימנית, הכלי יוזז לשם ויתקבל הלוח הבא:



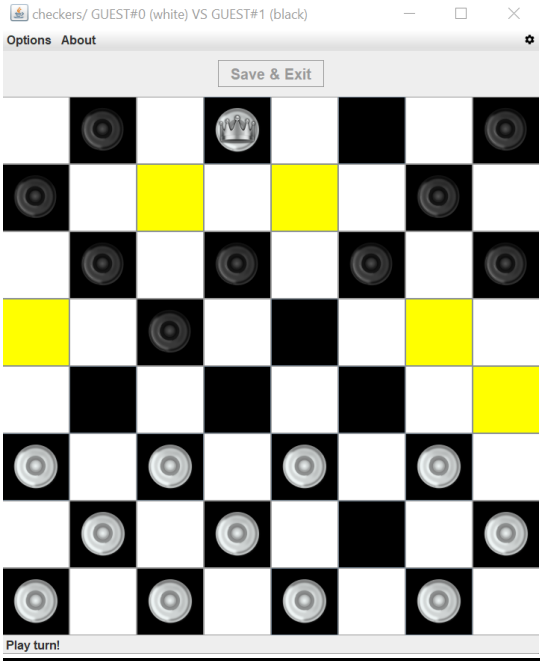
במידה ויש אפשרות של אכילה, החייל שיאכל יעלם מהלוח והכלי ידלג למשבצת הבאה:



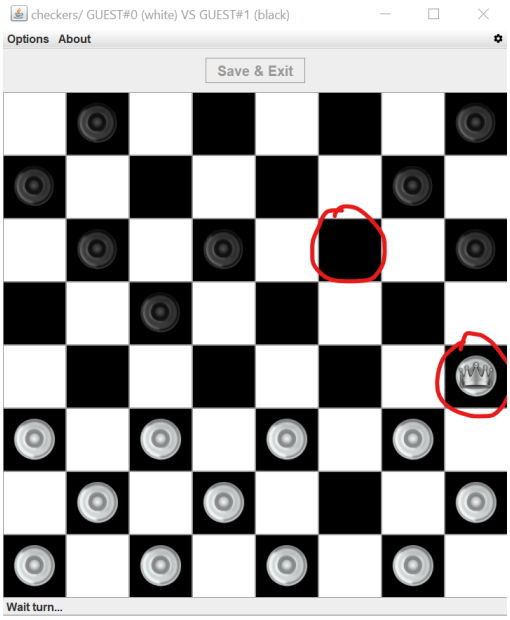
לאחר שנבחר באופציית האכילה (המשבצת השמאלית הצהובה) תתבצע האכילה:



כאשר כלי כלשהו מגיע לקצה השני של הלוח (במידה וזה כלי לבן אם הגיע לשורה הכי עליונה בלוח, במידה וכלי שחור לשורה התחתונה בלוח) הוא נהפך למלך. למלך אפשרויות תזוזה רבות יותר מאשר כלי רגיל (לפירוט נרחב עיין בסעיף 3.1 בחלק "חוקי המשחק ומהלכים"):

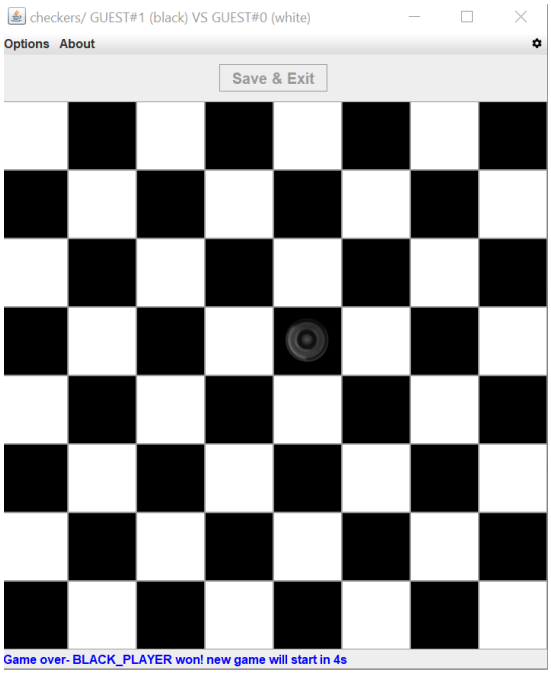


כמו שבאכילה דילוג מעל כלי יריב אוכל את השחקן, כך כאשר מדלגים עם מלך מעל כלי יריב כמה משבצות קדימה זה נחשב אכילה (לאו דווקא הגעה למשבצת שבאה ישר אחרי הכלי היריב). לדוגמה, אם נלך עם המלך למשבצת הצהובה הימנית ביותר, עדיין זה ייחשב אכילה והכלי השחור יאכל:



במשחק קיימת חובת אכילה. זאת אומרת, במידה וקיימת אפשרות לשחקן מסויים לאכול כלי יריב והוא במקום זה עושה מהלך אחר שאינו אכילה, הכלי שיכל לאכול איתו כלי יריב "נשרף" ויוצא מהמשחק.
במידה וכלי כלשהו ביצע אכילה והגיע למשבצת כלשהי וישנה מאותה המשבצת אפשרות לעשות עוד אכילה, ניתן לאכול את 2 הכלים היריבים (ניתן להמשיך ולבצע אכילות עד שמגיעים למשבצת ממנה אין עוד אפשרות אכילה). אין אפשרות לבצע את האכילות בצעד לאחור.

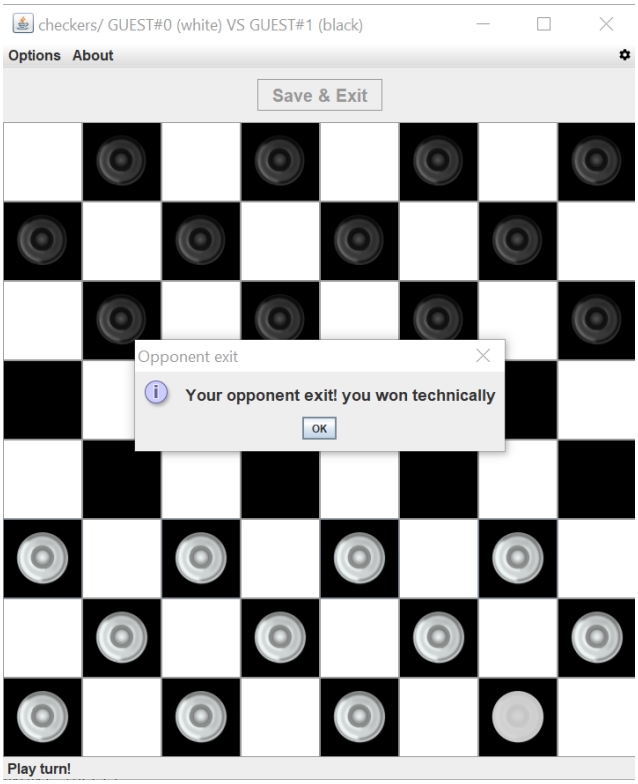
ניצחון יתקבל כאשר לשחקן אחד נותרו חיילים ולשני נגמרו כולם. בדוגמה הבאה התקבל ניצחון לשחקן עם הכלים השחורים:



תיקו יתקבל כאשר לשני השחקנים יש אותה כמות מלכים ובמשך 5 תורות לא התפתח כלום במשחק (לא נאכל חייל או מישהו ניצח).

לאחר שנגמר המשחק תתבצע ספירה לאחור של 7 שניות ולאחר שתסתיים יתחיל משחק חדש בין 2 השחקנים (כאשר השחקן שהתחיל ראשון במשחק הראשון יהיה שוב ראשון). אם אחד מהשחקנים הוא שחקן רשום, תישמר תוצאת המשחק במסד הנתונים לצורך הפקת סטטיסטיקות בהמשך.

במידה ובאמצע משחק אחד השחקנים התנתק מכל סיבה שהיא (נפילת חשמל, יציאה יזומה וכו') השחקן היריב מקבל הודעה שנגמר המשחק עקב התנתקות היריב והוא זוכה בניצחון טכני:



כפתור SAVE & EXIT- כפתור עבור שמירת משחק שלא נגמר נגד שחקן ממוחשב. בעת לחיצה על הכפתור תקפוץ הודעה לוודא שאנו אכן רוצים לשמור את המשחק. לאחר שאישרנו המשחק ייסגר ויישמר במסד הנתונים. במידה וניכנס עם אותו שחקן ששמרנו את המשחק ונבחר באפשרות משחק נגד שחקן ממוחשב, תקפוץ לנו האפשרות להמשיך את אותו משחק מהנקודה שנעצר כששמרנו אותו. כפתור זה פעיל כאשר מתחברים עם שחקן רשום ובוחרים באפשרות משחק נגד שחקן ממוחשב

תפריטים: קיימים 2 תפריטים עבור כל שחקן:

Options- תפריט שמתחלק ל-2 תתי תפריטים:

1. **Statistics-** תפריט ובו 4 אופציות לבקשה של סטטיסטיקות:

א. **All games played-** נותן את כל המשחקים של השחקן (היריב, מי המנצח ותאריך המשחק) ובתחתית סיכום קצר (כמה משחקים שוחקו סה"כ, נצחונות, תיקו והפסדים)

ב. **Games played last week-** בדומה לסעיף הקודם, רק שמציג את כל המשחקים שהשחקן שיחק בשבוע האחרון כולל הסיכום קצר

ג. **Games played VS AI-** מציג את כל המשחקים שהשחקן שיחק נגד שחקן יריב ממוחשב כולל סיכום קצר

ד. **Top5-** מציג את הטופ 5 של המשחק (5 השחקנים בעלי כמות הנצחונות הגדולה ביותר)

הערה: למעט טופ 5 שכל שחקן יכול לבקש, 3 האופציות הראשונות שמורות רק לשחקנים רשומים

2. **User options-** תפריט הכולל בתוכו אפשרויות משתמש:

א. **Delete saved games-** מוחק את כל המשחקים השמורים נגד AI שהשחקן שמר

ב. **Change password-** אפשרות לשנות את הסיסמא של השחקן

ג. **Forfeit-** שולח לשרת הודעה כי השחקן רוצה להפסיד ולצאת מהמשחק (מעין יציאה יזומה כמו לחיצה על איקס)

הערה: למעט forfeit שפתוח לכל השחקנים, 2 האופציות הראשונות שמורות אך ורק לשחקנים רשומים

About- תפריט הסבר על המשחק ובו 2 אפשרויות:

1. **game & rules-** הסבר על המשחק והחוקים שלו
2. **credits-** קרדיטים לאלו שעזרו בבניית הפרויקט

4. מדריך למתכנת

בחלק זה נסביר בצורה מעמיקה את קוד התוכנה מנקודת מבטו של המתכנת. נתחיל בדרישות ממערכת התוכנה, נמשיך עם פירוט מרכיבי הפרויקט שבהם נעשה שימוש בכתיבת הקוד כגון: ארכיטקטורת שרת/לקוח, פרוטוקול התקשורת TCP/IP, שקעים Sockets, תהליכונים Threads ומסד הנתונים. כמו כן יפורטו המחלקות, מבני הנתונים ואלגוריתמים לפעולות נבחרות, בהם נעשה שימוש בפרויקט.

4.1 הדרישות ממערכת התוכנה

- על המערכת להיות כתובה בשפת JAVA תוך שימוש בעקרונות תכנות מונחה עצמים מתקדם
- על המערכת להשתמש בארכיטקטורת שרת לקוח Client/Server
- על המערכת לבצע את התקשורת בין השרת ללקוח באמצעות Sockets עם פרוטקול TCP/IP
- על המערכת לטפל בחיבור מספר לא מוגבל של לקוחות ובמקביל (שימוש ב-Threads)

- בעת חיבור לקוח, על המערכת להציג מסך LOGIN שמאפשר ללקוח להתחבר משתמש רשום (עם שם משתמש וסיסמה) או כאורח. אם מתחבר כמשתמש רשום, נתוניו יבדקו מול מסד הנתונים.
- לאחר שלקוח התחבר (כמשתמש רשום או כאורח), על המערכת לאפשר לו לבחור את סוג השחקן מולו הוא ירצה לשחק: שחקן אנושי (לקוח אחר) או שחקן ממוחשב (רץ בשרת). אם בחר לשחק מול שחקן אנושי, הוא מקבל הודעה שעליו להמתין לחיבור לקוח נוסף שיהיה הבן זוג שלו למשחק. במקרה זה המשחק לא מתחיל עד אשר מתחבר עוד לקוח. אם בחר לשחק מול שחקן ממוחשב – המשחק מתחיל מייד.
- על המערכת לאפשר מספר לא מוגבל של משחקים. כל משחק הוא בין זוג שחקנים.
- כאשר משחק מסתיים, המערכת תודיע לשחקנים על סיום המשחק ותוצאתו (מי ניצח/הפסיד או תיקו) ותאפשר התחלת משחק חדש ע"י ספירה לאחור.
- אם משחק הוא בין שני שחקנים אנושיים (שהם שחקני רשת – לקוחות) ולפחות אחד ביצע התחברות כמשתמש רשום, אז תוצאות המשחקים שלהם ישמרו במסד הנתונים לצורך הפקת סטטיסטיקות עבור כל שחקן.
- על המערכת לשמור במסד נתונים לפחות 3 טבלאות: משתמשים רשומים (שם משתמש,סיסמה) משחקים (משתמש1, משתמש2, זמן המשחק-תאריך שעה, תוצאת המשחק) ועוד...
- בכל שלב לקוח יוכל להתנתק ע"י סגירת החלון שלו. על המערכת לזהות זאת ולבדוק: אם מדובר על לקוח שהוא באמצע משחק, אז יש להודיע ללקוח השני (אם הוא אנושי) על סיום המשחק ולבצע סגירה והפסקת המשחק בצורה מסודרת (הקפצת הודעות מתאימות, סגירת סוקטים וכו').
- על המערכת לזהות ניתוקים לא צפויים (כגון: נפילת לקוח/שרת) ולהקפיץ הודעות מתאימות ולבצע סגירה מסודרת.

4.2 ארכיטקטורת המערכת

בסעיף זה נסביר על ארכיטקטורת המערכת המתארת את מרכיבי התוכנה שבהם נעשה שימוש לצורך כתיבת קוד הפרויקט. עבור כל מרכיב נסביר למה הוא משמש? למה היה צריך אותו במימוש הפרוייקט? למה בחרנו דווקא בו? ועוד.

4.2.1 מודל שרת-לקוח Client-Server Model

מודל שרת לקוח הינו מודל בו קיים שרת במקום כלשהו שתפקידו לשרת את הלקוחות שמתחברים אליו ולבצע את מבוקשם. השרת בדרך כלל פאסיבי ומחכה לחיבור של הלקוחות ושליחת בקשות מצידם, לפיהם הוא מגיב אליהם בחזרה. התקשורת בין השרת ללקוחות מבתצעת כאשר הלקוח שולח בקשה לשרת לקבל ממנו דברים כלשהם, והשרת מצידו מחזיר תגובה עם הדבר המבוקש.
השתמשנו במודל שרת לקוח כיוון שרצינו לממש משחק מרובה משתתפים כך שהרבה לקוחות יוכלו להתחבר למשחק ולשחק במקביל.

4.2.2 פרוטקול תקשורת TCP/IP

פרוטוקול TCP/IP הוא פרוטוקול תקשורת. הפרוטוקול מאפשר העברת מידע אמינה ויעילה בין 2 ישויות ברשת. בניגוד לUDP שתפקידו להעביר נתונים בזמן אמת ואם חלק מהנתונים נפלו בדרך הוא לא חוזר להביא אותם, פרוטוקול TCP מבצע את העברת המידע בצורה יעילה וגם אם "נפלו" נתונים בדרך, הוא דואג לחזור להביא אותם. בחרנו בפרוטוקול תקשורת זה בפרויקט כיוון שיש חשיבות גדולה לכך שכל הנתונים יגיעו במלואם לצד השני (אם נופל פרט אחד בדרך זה יכול לשבש את המשחק. לדוגמה: אם נפל בדרך המהלך ששחקן מסויים החליט לבצע בתורו או אפילו פרט קטן מתוכו המהלך לא יקרה כמו שצריך ותהיה תקלה במשחק).

4.2.3 שקעים Sockets

סוקט זה שקע שמשמש להעברת מידע בין 2 ישויות במרחב האינטרנט. כל שקע מורכב מ-2 חלקים: אינפוט סטרים (IS) שתפקידו לקבל הודעות, ואאוטפוט סטרים (OS) שתפקידו לשלוח הודעות לישויות אחרות באינטרנט. השתמשנו בסוקט בפרויקט כדי להעביר מידע בין השרת ללקוחות (כגון מהלך ששחקן ביצע עובר לשרת דרך הסוקט והשרת מעביר ליריב של אותו השחקן את הלוח המעודכן עם המהלך שנעשה דרך הסוקט). היה בו צורך בפרויקט על מנת להעביר נתונים בין השרת והלקוחות, שכן בלעדיו לא ניתן היה ליצור בכלל תקשורת עם השרת ולהפעיל את המשחק.

4.2.4 מסד הנתונים

מסד הנתונים הוא מקום שבו אנו אוגרים את הנתונים שקשורים למשחק. הוא שמור על השרת וניגשים אליו דרך ה- drivers וה- getConnection במחלקה DB. יש בו צורך כדי לבצע כל מיני סוגי פעולות כמו שליפת סטטיסטיקות והמשך משחקים שנשמרו נגד AI.
קיימות 3 טבלאות במסד הנתונים:

1. **Users-** תפקידה לשמור בתוכה נתונים של המשתמשים הרשומים הקיימים. נתונים כמו: שם משתמש, סיסמה, גיל וכו'.

un	pw	age	email
a	1	20	a@gmail.com
ariel	1111	20	ariel@gmail.com
david	2222	17	david@gmail.com
moshe	4444	35	moshe@gmail.com
nofar	5555	22	nofar@gmail.com
rivka	6666	40	rivka@gmail.com
ronen	7777	16	ronen@gmail.com

עמודות בטבלה:

Un- עמודה שאוגדת בתוכה את כל שמות המשתמשים. המשתנה הוא טיפוס מסוג מחרוזת קצרה (varchar) והיא עמודת המפתח של הטבלה.

Pw- עמודת שאוגדת בתוכה את כל הסמאות של המשתמשים. המשתנה הוא טיפוס מסוג מחרוזת קצרה (varchar) והיא אינה עמודת מפתח.

Age- עמודת שאוגדת בתוכה את כל הגילאים של המשתמשים. המשתנה הוא טיפוס מסוג מספר שלם (int) והיא אינה עמודת מפתח.

Email- עמודת שאוגדת בתוכה את כל הדוא"ל של המשתמשים. המשתנה הוא טיפוס מסוג מחרוזת קצרה (varchar) והיא אינה עמודת מפתח.

2. **הטבלה games:**

autoID	un1	un2	winner	datetime
1	ilan	itay	itay	28.10.2020 11:50
2	ilan	ariel	ariel	28.10.2020 21:30
3	itay	ariel	itay	28.10.2020 13:00
4	ariel	moshe	moshe	28.10.2020 17:25
5	GUEST#0	a	a	1642413751353
6	ronen	AI	ronen	1642446893091
7	ronen	AI	AI	1642447010528
8	a	AI	AI	1644780431763
9	a	AI	AI	1644780505798
10	a	AI	AI	1644780505806
11	rivka	AI	AI	1644780598351

הטבלה games מכילה מידע על משחקים שנגמרו. כל משחק שהסתיים ולפחות אחד מהשחקנים ששיחק היה משתמש רשום, נשמר בטבלה הסטטוס שלו: מי השחקנים, המנצח ביניהם ותאריך המשחק (במילי שניות). הטבלה משמשת כדי שנוכל לאחר מכן לשלוף סטטיסטיקות עבור שחקן מסויים. לדוגמה: כל המשחקים ששיחק, כמה נצחונות, כמה שיחק נגד AI וכו'

עמודות הטבלה:

autoID- מספור אוטומטי שמזהה את המשחק (מספר זהות של המשחק). העמודה מכילה טיפוס מסוג מספר אוטומטי והיא עמודת המפתח של הטבלה.

Un1- מייצגת את השחקן הראשון שהתחבר מתוך השניים ששיחקו. העמודה מכילה טיפוס מסוג טקסט קצר (varchar) והיא אינה עמודת מפתח.

Un2- מייצגת את השחקן השני שהתחבר מתוך השניים ששיחקו. העמודה מכילה טיפוס מסוג טקסט קצר (varchar) והיא אינה עמודת מפתח

winner- מייצגת את השחקן שניצח במשחק. העמודה מכילה טיפוס מסוג טקסט קצר (varchar) והיא אינה עמודת מפתח

datetime- מייצגת את התאריך בו המשחק שוחק (במילי שניות). העמודה מכילה טיפוס מסוג טקסט קצר (varchar) והיא אינה עמודת מפתח

3. **הטבלה unfinishedGames:**

auto	un	board	playerTurn	dateTime
12	nofar	----------	AI	46162718246
18	nofar	----------	AI	46220192440

הטבלה מכילה נתונים על משחקים נגד AI שנשמרו על מנת להמשיכם בעתיד. כאשר שחקן בוחר לשמור משחק נשמרים 5 פרטים על המשחק:

Auto- מספור אוטומטי שמייצג מספר מזהה של המשחק. העמודה היא טיפוס מסוג מספור אוטומטי והיא עמודת המפתח של הטבלה

Un- מייצג את שם המשתמש של השחקן שבחר לשמור את הלוח. העמודה היא טיפוס מסוג מחרוזת קצרה (varchar) והיא אינה עמודת המפתח של הטבלה.

Board- מייצג את הלוח שנשמר. העמודה היא טיפוס מסוג מחרוזת ארוכה והיא אינה עמודת המפתח של הטבלה.

playerTurn- מייצג את שם השחקן שתורו לשחק בנקודה בה ימשיך המשחק. העמודה היא טיפוס מסוג מחרוזת קצרה (varchar) והיא אינה עמודת המפתח של הטבלה

dateTime- מייצג את התאריך בו נשמר המשחק. העמודה היא טיפוס מסוג מחרוזת קצרה (varchar) והיא אינה עמודת המפתח של הטבלה

4.2.5 תהליכונים Threads

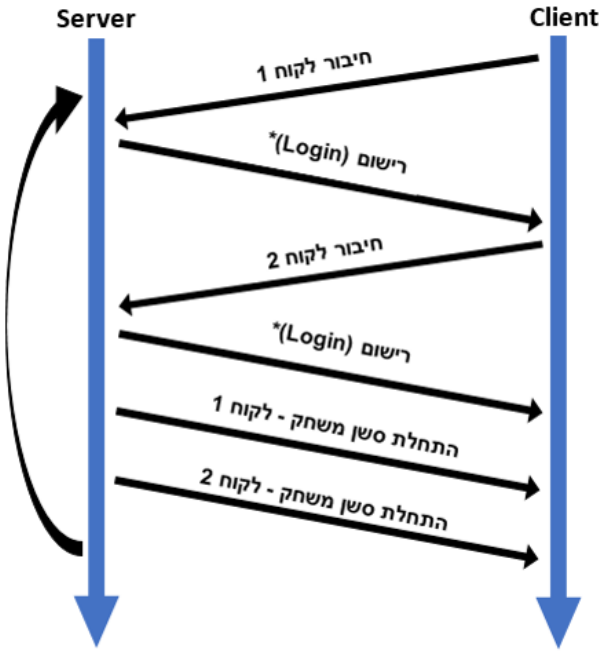
תהליכונים אלו תהליכים קטנים שרצים במקביל בתוך תהליך גדול. השתמשנו בהם בפרויקט בין היתר בהרצת משחקים במקביל, כאשר כל משחק בפני עצמו הוא תהליכון. יש צורך בתהליכונים כיוון שיש צורך במקביליות במשחק- מכיוון שהרבה שחקנים משחקים במקביל לא ניתן לעבוד בצורה סדרתית כי אז זה יתקע את כל השאר עד שיגיע תורם בסדר הקוד- כל משחק הוא ישות בפני עצמה שצריכה לרוץ בפני עצמה בלי קשר לשאר המשחקים שמשוחקים גם כן.

4.3 תהליכים עיקריים וזרימת המידע

בסעיף זה נסביר את התהליכים העיקריים שמהם מורכבת מערכת התוכנה ואת זרימת המידע ביניהם. את התהליכים נציג בצורה גרפית באמצעות **תרשימי רצף (Sequence Diagrams)** שיתארו את זרימת המידע (פרוטוקולי השיח) בין השרת ללקוח החל מחיבור הלקוחות, שידוך בין שני שחקנים להתחלת משחק, התנהלות ומהלך המשחק ביניהם, סיום משחק ועוד.

4.3.1 חיבור לקוחות והתחלת סשן משחק

להלן דוגמה לתרשים רצף עבור **התהליך הראשי** "חיבור לקחות והתחלת סשן משחק":

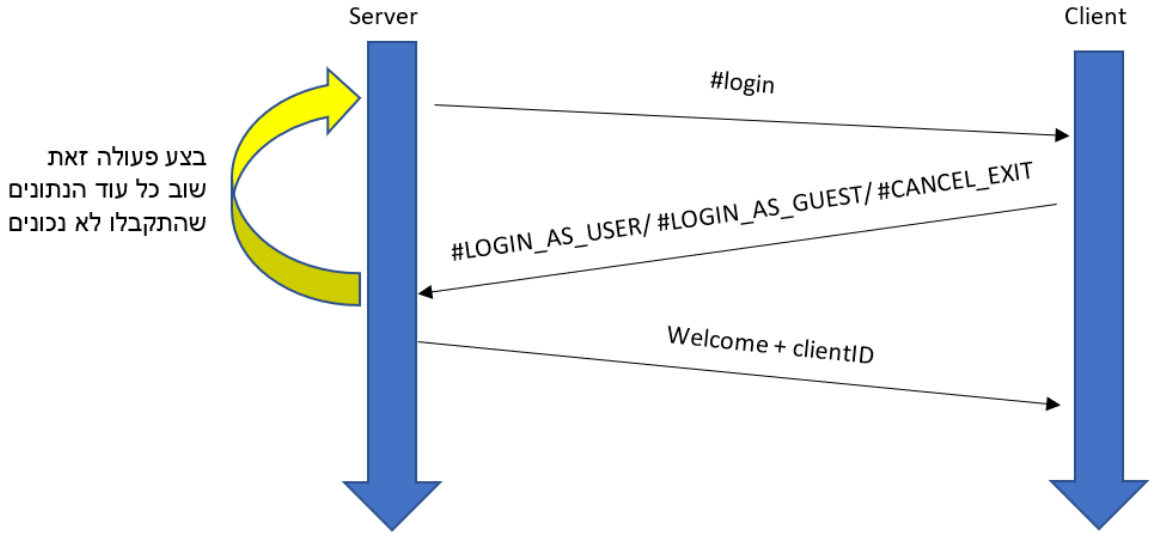


תרשים הרצף הנ"ל מתאר את פרוטוקול השיח בין השרת ללקוחות, עבור התהליך הראשי בו השרת ממתין לחיבור לקוחות, בלולאה אינסופית, ובעבור כל זוג לקוחות שהתחברו בהצלחה, הוא משדך ביניהם ויוצר עבורם משחק.

שני החצים האנכיים, מתארים שני הצדדים בשיח - השרת (משמאל) והלקוח (מימין) - שמהווים את ציר הזמן. החצים השחורים האופקיים, הם ההודעות שעוברות מצד לצד, וראש החץ מעיד על כיוונם. מעל החצים יש כיתוב המתאר את סוג ההודעה הנשלחת ומידע נוסף שמועבר עם ההודעה.

4.3.2 התחברות לקוח

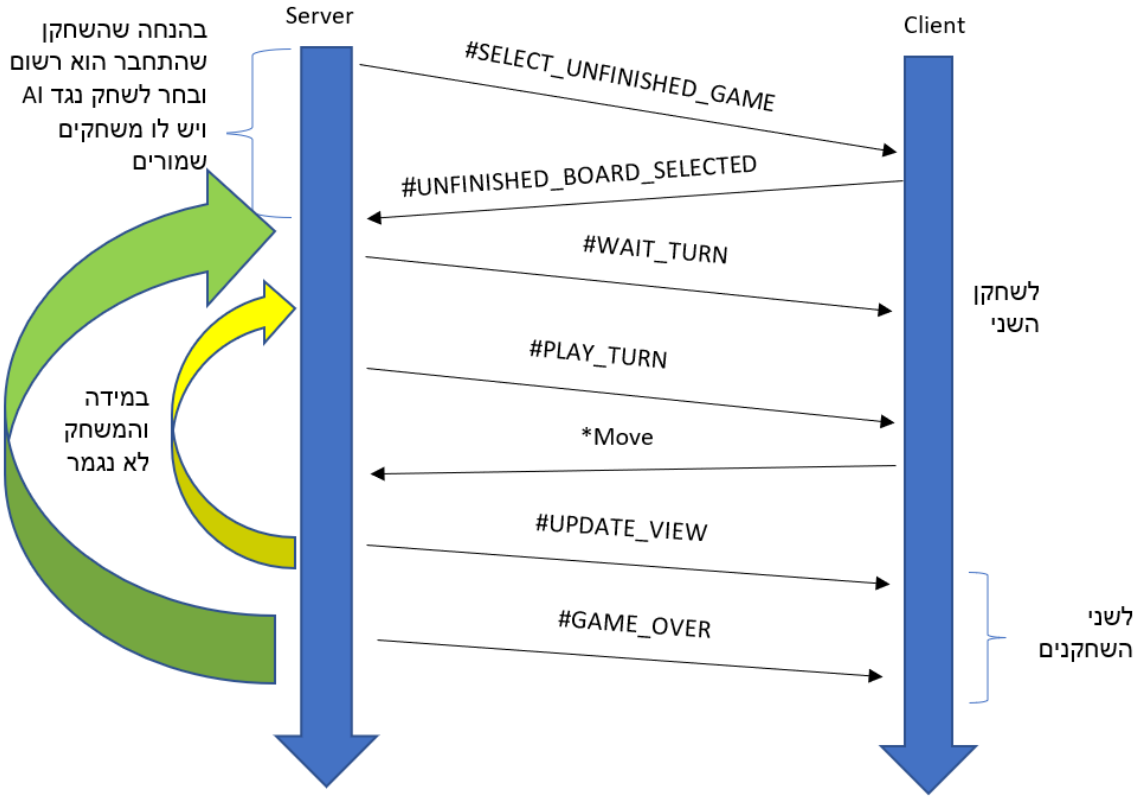
להלן דוגמה לתרשים רצף עבור תת-תהליך "**התחברות לקוח - Login**":



התרשים הנ"ל מתאר התחברות של לקוח כלשהו. כאשר הלקוח מתחבר לשרת על ידי הזנת כתובת IP והפורט עליו רץ השרת, נוצר שקע התקשורת ביניהם (Socket), דרכו השרת שולח ללקוח הודעת #LOGIN, שפירושה הוא שהלקוח יקפיץ את חלון הלוגין לצורך הזדהות מול השרת. לאחר שהלקוח מילא את הפרטים ושלח אותם, השרת בודק שהפרטים אכן נכונים (בהנחה שהתחבר כמשתמש רשום, שאז יש צורך לבדוק את הפרטים). במידה והפרטים נכונים או שהשחקן התחבר כאורח, תהליך ההזדהות הושלם בהצלחה ונשלחת לשחקן הודעת welcome. במידה והפרטים לא נכונים או שהשחקן כבר מחובר לשרת, תישלח הודעת שגיאה והחלון יקפוץ שוב.

4.3.3 סשן משחק בין 2 שחקנים

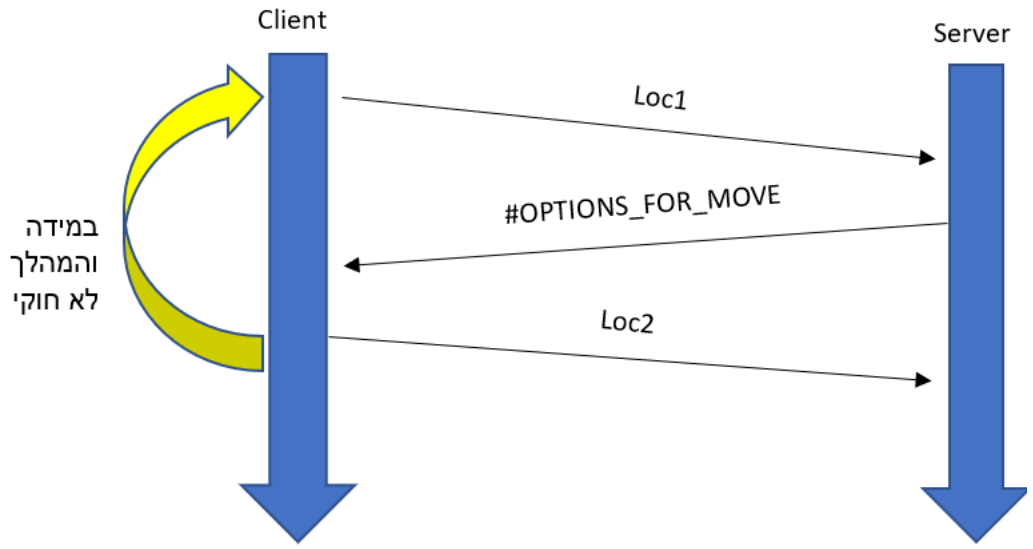
להלן דוגמה עבור התהליך הראשי **"סשן משחק בין 2 שחקנים":**



* תת התהליך יוסבר בהמשך

בתרשים הנ"ל מתואר סשן משחק כלשהו. במידה והתחבר שחקן רשום, בחר לשחק נגד שחקן ממוחשב ויש לו משחקים שמורים, התוכנית תקפיץ לו אופציה לבחור משחק שמור כלשהו או לחלופין להתחיל משחק חדש. לאחר מכן מתחיל סשן המשחק. לשחקן שמתחיל מופיע בתווית התחתונה הודעה שמודיעה לו שעכשיו תורו לשחק, ואילו לשחקן השני הודעה שעליו לחכות לתורו. השחקן שעכשיו תורו שולח מהלך כלשהו שבחר, השרת מעדכן את המודל של המשחק (הלוח הלוגי) עם המהלך הנבחר ושולח הודעה לשני השחקנים לעדכן את לוח המשחק המוצג לפניהם. במידה ונגמר המשחק, נשלחת הודעה לשני השחקנים שהמשחק נגמר עם זהות המנצח, תתבצע ספירה לאחור ויתחיל משחק חדש (במידה ושיחק שחקן רשום אחד לפחות, נתוני המשחק ישמרו במסד הנתונים). במידה והמשחק לא נגמר, יתחלפו תורות השחקנים והמשחק ימשיך על הסדר הנ"ל.

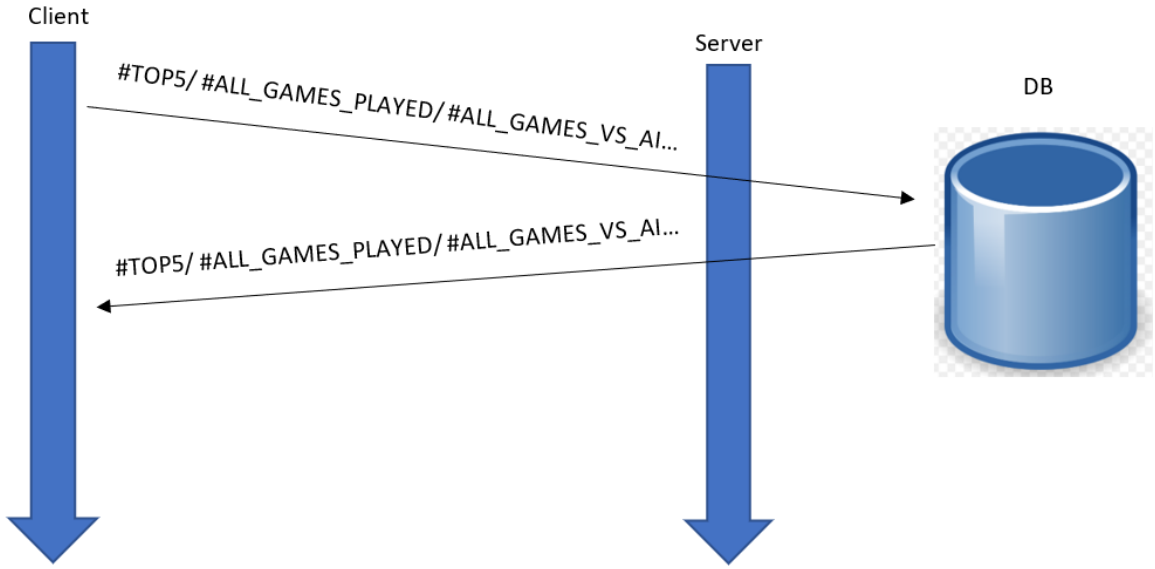
להלן תת התהליך "**בחירת מהלך**":



בתחילה, השחקן שעכשיו תורו לוחץ על כלי מסוים שברצונו להזיז. לאחר מכן, ישלחו לשחקן כל אפשרויות התזוזה האפשריות לכלי שבחר. צד הלקוח צובע בצהוב את כל המשבצות אליהן ניתן להזיז את הכלי. לאחר שהשחקן בחר משבצת כלשהי, השרת בודק את תקינותה: במידה ונבחרה משבצת חוקית, המהלך יתבצע על הלוח הלוגי ולאחר מכן גם הלוחות הוויזואליים של השחקנים יתעדכנו בהתאם (כמבואר בתרשים הראשי "סשן משחק"). אם המהלך לא חוקי, כגון שהשחקן לחץ שוב על הכלי כדי לבטל את הכלי שנבחר ולבחור חדש, התהליך יתחיל מחדש לפי הסדר הנ"ל.

4.3.4 בקשת סטטיסטיקה מהשרת

להלן התהליך **"בקשת סטטיסטיקה מהשרת":**



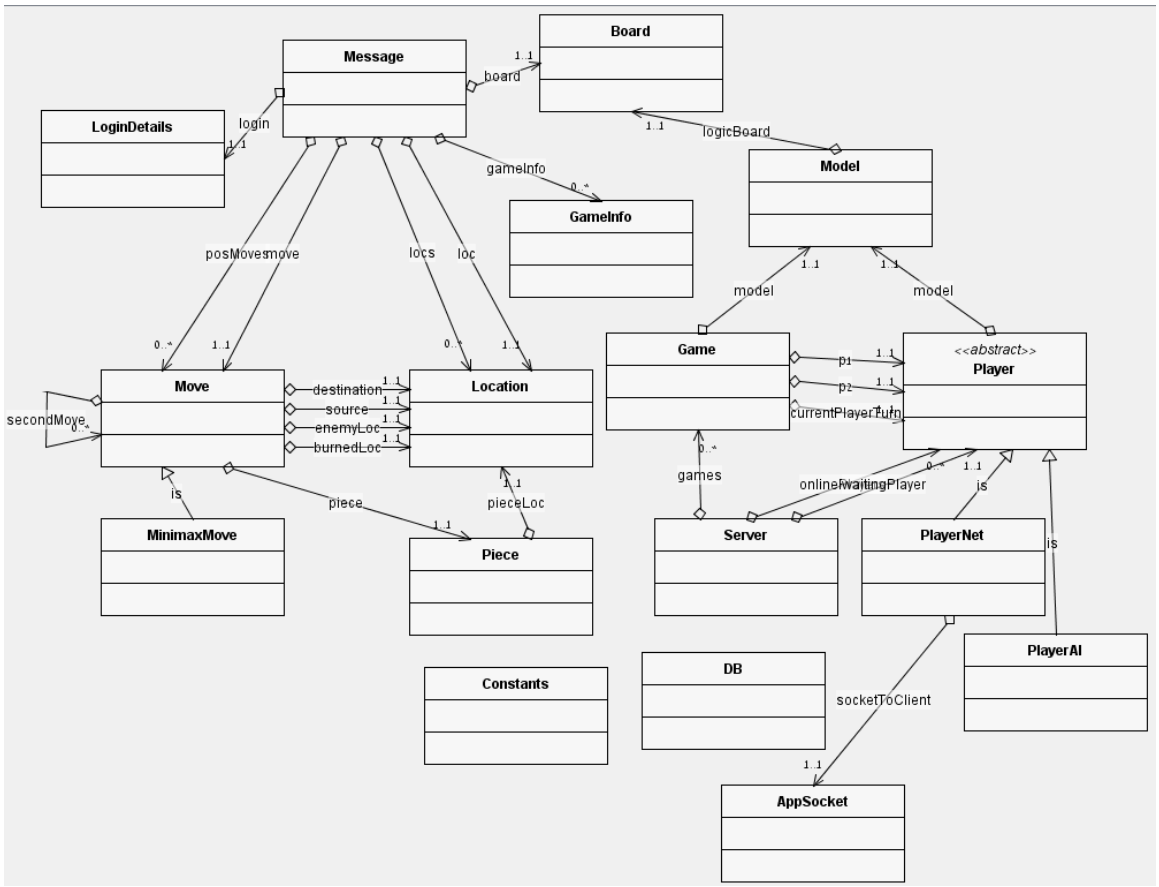
בתרשים הנ"ל השחקן שולח בקשה לשרת עבור סטטיסטיקה כלשהי (יש עוד אפשרויות חוץ ממה שצוין בתרשים. ניתן לראות לעיל בסעיף 3.3.3 את הפירוט המלא). השרת מקבל את הבקשה וניגש לשרת לשלוף ממנו את המידע הרלוונטי. לאחר שהשרת שלף את המידע הוא מחזיר אותו לשחקן, והוא מציג אותו בחלונית.

4.4 מחלקות הפרויקט

בסעיף זה נציג את כל המחלקות של הפרויקט. הפרויקט מורכב משני מודולים : צד שרת וצד לקוח. לכל מודול יש מחלקות ייחודיות רק לו וכאלו המשותפות לשניהם. להלן אציג פירוט עבור צד השרת וצד הלקוח.

4.4.1 תרשים UML של המחלקות בצד השרת

להלן תרשים ה- UML של צד השרת, הבנוי מ-17 מחלקות שעליהן אכתוב פירוט מעמיק. צד השרת בנוי מ-9 מחלקות המשותפות לו וללקוח וכן 8 מחלקות ייחודיות רק לו. כדי להריץ את השרת צריך להריץ את המחלקה server.



4.4.2 תיעוד המחלקות בצד השרת

1. **-DB** זו מחלקת שירות שתפקידה לגשת את ה- DB ולשלוף ממנו מידע כמו סטטיסטיקות או לעדכן אותו כמו שינוי ססמה.

```
+ final String APP_DB_FILE_PATH

+ static Connection getConnection(String dbFileRelativePath)
+ static ResultSet runQuery(String sql, String dbFilePath)
+ static int runUpdate(String sql, String dbFilePath)
+ static void printResultSet(ResultSet rs, int colWidth)
+ static String toStringResultSet(ResultSet rs, int colWidth)
- static String makeFormatDate(String date)
+ static boolean isUserExists(String un, String pw)
+ static ArrayList<GameInfo> getAllPlayers()
+ static void saveBoardToDB(String ID, String board, String playerTurn)
+ static ArrayList<String> selectUnfinishedBoards(String un)
+ static void gameOver(String p1, String p2, String pWinner)
+ static ArrayList<GameInfo> allGamesPlayed(String un)
- static String totalWins(String un)
- static String totalTies(String un)
- static String totalLoses(String un)
- static String totalGamesPlayed(String un)
+ static ArrayList<GameInfo> top5()
+ static String deleteUser(String un)
+ static void deleteGamesVsAI(String un)
+ static void changePassword(String un, String pw)
+ static String deleteSavedGames(String un)
+ static String getPlayerTurn(String dateBoard)
+ static String getBoard(String un, String dateTime)
+ static void deleteUnfinishedGameContinued(String date)
+ static ArrayList<GameInfo> gamesPlayedLastWeek(String un)
+ static ArrayList<GameInfo> gamesVsAI(String un)
+ static void main(String arg)
```

```java
/**
 * Checks if user with un and pw is exists in Users table
 *
 * @param un user name of user
 * @param pw password of user
 * @return true if user exists, false otherwise
 * @throws SQLException if the func can't run the query
 */
public static boolean isUserExists(String un, String pw) throws SQLException
```

```java
/**
 * save board to unfinishedGames table (register user vs AI player)
 *
 * @param ID of player that saved the game
 * @param board that the func need to save
 * @param playerTurn player that start play when game will continue
 */
public static void saveBoardToDB(String ID, String board, String playerTurn)
```

```java
/**
 * save game status when game over or user disconnected (save game status
 * only if at least one register)
 *
 * @param p1 player 1 in game
 * @param p2 player 2 in game
 * @param pWinner the winner in the game
 */
public static void gameOver(String p1, String p2, String pWinner)
```

2. **Game -** מחלקה המייצגת משחק בין 2 שחקנים- אנושי מול אנושי או אנושי מול ממוחשב

```
-Player p1
-Player p2
-Model model
-Player currentPlayerTurn
-boolean gameStopped

+Game(Player player1, Player player2)
+Game(Player player1, Player player2, Model model, Player currentPlayerTurn)
+Player getP1()
+Player getP2()
+Player getCurrentPlayerTurn()
+void setGameStopped(boolean gameStopped)
+void runGame(boolean continueGame)
-void defineStartGame()
-void defineContinueGame()
-boolean checkGameStatus(boolean swithTurn)
-Player getOpponent()
-void switchTurn()
+boolean registerPlayerPlayed()
+void stopRunGame()
+void playerDisconnected(String un)
-void saveGamePlayerDisconnected()
+String toString()
```

```java
/**
 * Makes the game to start running
 *
 * @param continueGame if game is saved game
 */
public void runGame(boolean continueGame)
```

```java
/**
 * Constructor for continued game between player 1 and AI
 *
 * @param player1 player1 in game
 * @param player2 player2 in game
 * @param model the model of game
 * @param currentPlayerTurn player that his turn to play
 * when game will start
 */
public Game(Player player1, Player player2, Model model,
```

```java
/**
 * Constructor for new game between 2 players
 *
 * @param player1 player1 in game (will play first)
 * @param player2 player2 in game
 */
public Game(Player player1, Player player2)
```

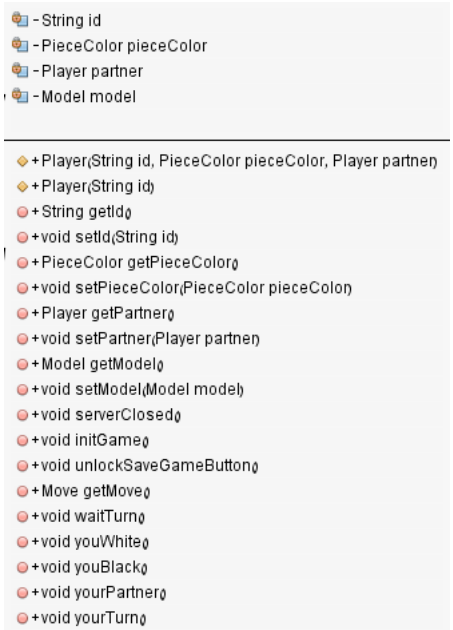
3. **MinimaxMove-** מחלקה המייצגת מהלך מינימקס עם עומק וניקוד. המחלקה יורשת מהמחלקה Move כך שהיא בעצם מוב משודרג.

- int grade
- int depth
+ final int EMPTY

+ MinimaxMove(int grade, int depth, Piece player, Location source, Location dest, boolean isEat, Location enemyLoc, Location burnedLoc)
+ MinimaxMove(MinimaxMove other)
+ MinimaxMove()
+ int getGrade()
+ void setGrade(int grade)
+ int getDepth()
+ void setDepth(int depth)
+ boolean isDepthLegit()
+ String toString()

```
/**
 * Copy constructor
 *
 * @param other minimaxMove that copy his
 * values from him
 */
public MinimaxMove(MinimaxMove other)
```

4. **Player-** מחלקה אבסטרקטית (לא ניתן ליצור ממנה עצם- מחלקה כללית עבור תת מחלקות כלשהם). מייצגת שחקן כלשהו במשחק (אנושי או ממוחשב).




```
/**
 * Sends to player request to make move
 *
 * @return move of player selected
 */
public abstract Move getMove();
```

```
/**
 * Constructor for new player
 *
 * @param id of player
 * @param pieceColor white or black pieces
 * @param partner who is his opponent
 */
public Player(String id, PieceColor pieceColor, Player partner)
```

5. **PlayerAI-** מחלקה המייצגת שחקן AI ממוחשב. המחלקה יורשת מהמחלקה האבסטרקטית Player (כיוון שהיא סוג של שחקן). כאשר שחקן אנושי כלשהו בוחר לשחק נגד AI, נוצר עצם מהמחלקה, מולו משחק השחקן האנושי.

```
- int minimaxTimeoutInSec

+PlayerAI(int minimaxTimeout, String id)
+PlayerAI()
+int getMinimaxTimeout()
+void setMinimaxTimeout(int minimaxTimeout)
+void gameOverTie()
+String toString()
+Move getMove()
+void waitTurn()
+void gameOver(PieceColor player)
+void yourTurn()
+void initGame()
+void updateView(Move move)
+void youWhite()
+void youBlack()
+void countDownFinished()
+void unlockSaveGameButton()
+void yourPartner()
+void yourOpponentExit()
+void selectUnfinishedGames(ArrayList<String> boards)
+void unlockRegisterOptions()
+void serverClosed()
```

```
/**
 * Constructor
 *
 * @param minimaxTimeout how many time minimax
 * run for search move
 * @param id of playerAI
 */
public PlayerAI(int minimaxTimeout, String id)
```

6. **PlayerNet-** מחלקה המייצגת שחקן אנושי. המחלקה יורשת מ- Player (כיוון שהיא סוג של Player)

```
/**
 * Constructor to create new playerNet
 *
 * @param socketToClient socket to the player (for read and write messages)
 * @param withAI if the player pis playing against AI
 * @param id of player
 */
public PlayerNet(AppSocket socketToClient, boolean withAI, String id)
```

7. **Model-** מחלקה האחראית על הלוגיקה של המשחק (כמו אפשרויות התזוזה של הכלים).

```
-final int ROWS
-final int COLS
+final int MINIMAX_TIME
-Board logicBoard
-int numMoves

+Model()
+Model(Model model)
+void setModel(Board logicBoard)
+Board getLogicBoard()
+ArrayList<Location> getEmptyLocs()
-boolean isWhiteTile(Location loc)
+void setup()
-void normalSetup()
-void kingsMovesTest()
-void testingSetup()
-void smallSetup()
-void doubleEatingSetup()
-void tripleEatingSetup()
+boolean playerCantMove(PieceColor player)
+Piece pieceInLoc(Location loc)
+Piece pieceInLoc(Board board, Location loc)
+Board makeMove(Board board, Move move)
-Board makeMoveOnCopyBoard(Board board, Move move)
+Board makeMove(Move move)
+void updateMove(Move move)
+void updateMove(Board board, Move move)
```

הערה: קיימות עוד פונקציות שלא הוכנסו כאן. ניתן לצפות בהם בקוד עצמו

```java
/**
 * Gets best move by minimax (for AI player)
 *
 * @param player the player the func search for him a move
 * @param board the board that func check on him a move
 * @param isMax if func search max or min grade (current player
 * or his opponent)
 * @param depth the depth that we want to stop the recursion
 * @param alpha to check if need to do alpha pruning
 * @param beta to check if need to do beta pruning
 * @return the best move to make that could be found to a specific depth
 */
private MinimaxMove minimax(PieceColor player, Board board, boolean isMax,
        int depth, int maxDepth, int alpha, int beta)
```

```java
/**
 * Gets grade to player for specific board
 *
 * @param board the board the func evaluate
 * @param player the player for him the board evaluated
 * @return evaluation for player on a specific board
 */
public int eval(Board board, PieceColor player)
```

```java
/**
 * Gets all possible moves that player with pieceColor playerSign can do on
 * specific board
 *
 * @param board board that search on him moves to player
 * @param playerSign player for him the func search moves
 * @return all possible moves for a player on specific board
 */
public ArrayList<Move> getAllPossibleMoves(Board board, PieceColor playerSign)
```

8. **Server-** מחלקה המייצגת את השרת. דרכה מריצים את השרת אליו יתחברו הלקוחות. השרת גם מקבל הודעות מהלקוחות שלו ופועל ומגיב להם בהתאם, לדוגמה : מקבל מלקוח בקשה לסטטיסטיקה, ניגש למסד הנתונים, שולף אותה ומחזיר ללקוח את התוצאה.

```
+final String SERVER_WIN_TITLE
-final Dimension SERVER_WIN_SIZE
-final Color SERVER_LOG_BGCOLOR
-final Color SERVER_LOG_FGCOLOR
+final Font SERVER_LOG_FONT
-JFrame frmWin
-JTextArea areaLog
-String serverIP
-int serverPort
-boolean serverSetupOK
-ServerSocket serverSocket
-int autoClientID
-ArrayList<Player> onlinePlayers
-ArrayList<Game> games
-Player waitingPlayer
---
+Server()
+void runServer()
-AppSocket waitForClient()
-void handleClient(AppSocket socketToClient)
-void createNewGameAndRun(Player player1, Player player2)
-void sendUnfinishedGamesToChoose(Player player)
+PlayerNet login(AppSocket clientSocket)
-void handleCmdSocket(PlayerNet player)
-void processCmdFromClient(Message msg, PlayerNet player)
+void continueUnfinishedGameAndRun(Player player, Model model, String playerTurn)
-void playerSavedGame(Player player)
-void playerDisconnected(Player player)
+void makeForfeit(String un)
-String getPlayerTurnByPlayer(String playerID)
-void sendYourOpponentExit(Player player)
-boolean playerIsOnline(String ID)
```

הערה: קיימות עוד פונקציות במחלקה. ניתן לצפות בהם דרך הקוד

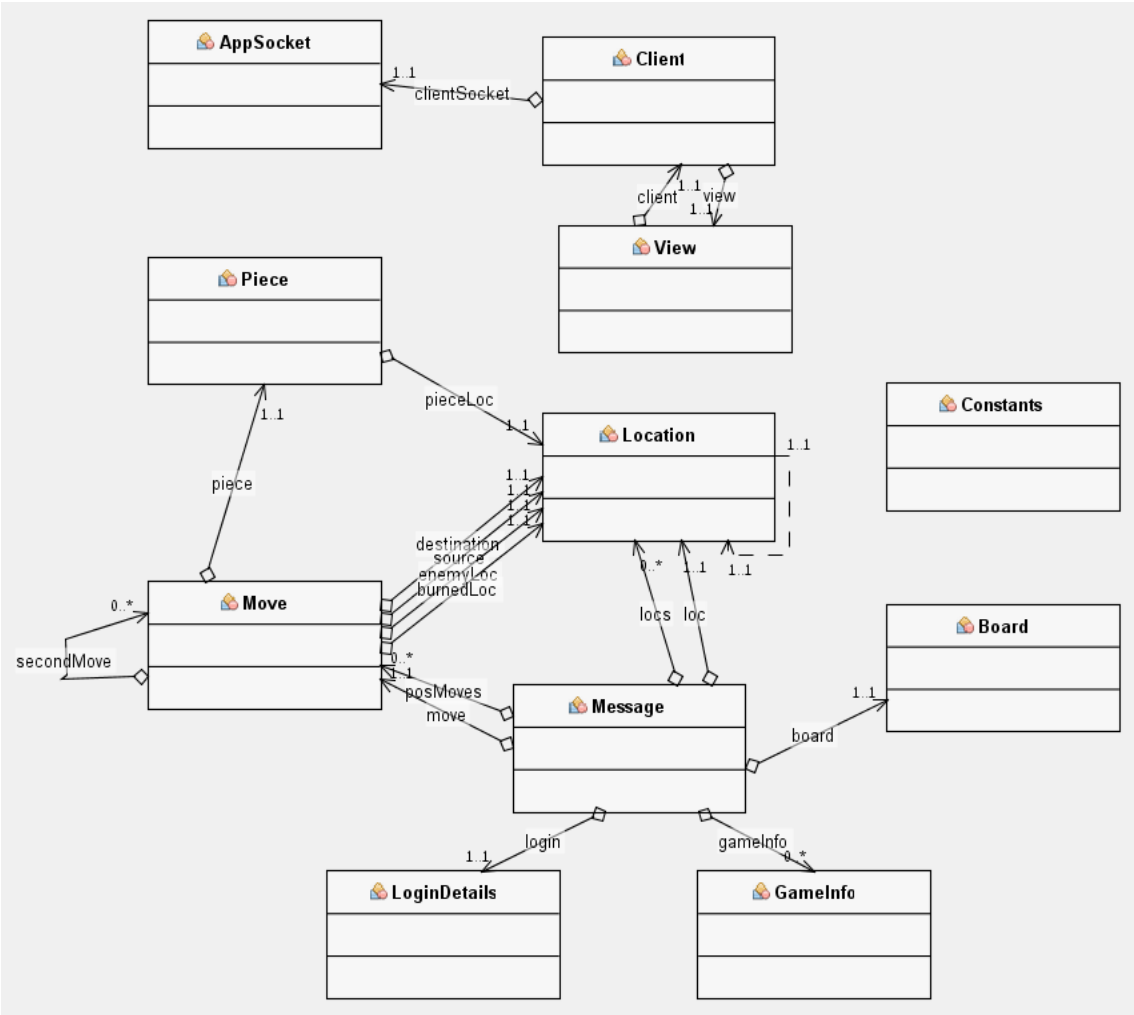
```
// Enter the 2 players to new game and starts
// the running of the game
private void createNewGameAndRun(Player player1, Player player2)


// handle client in a separate thread
private void handleClient(AppSocket socketToClient)


/**
 * Gets new playerNet that made login successfully
 *
 * @param clientSocket AppSocket to client (to send and receive messages)
 * @return PlayerNet from the login of client. null if client choosed cancel
 */
public PlayerNet login(AppSocket clientSocket)
```

4.4.3 תרשים UML של המחלקות בצד הלקוח

להלן תרשים UML של צד הלקוח. צד הלקוח בנוי מ- 10 מחלקות כאשר 9 מתוכן משותפות לו ולשרת ועוד 2 ייחודיות רק לו. על מנת להריץ את הלקוח יש להריץ את המחלקה client.



4.4.4 תיעוד המחלקות בצד הלקוח

Client- מחלקה המייצגת לקוח. מריצים את המחלקה ודרכה מתחברים לשרת ומתחילים לשחק. צד הלקוח הוא זה שמעביר הודעות לשרת ומקבל הודעות ממנו ופועל לפיהן (לרוב מבצע פעולות על לוח התצוגה שלו- view).

הערה: קיימות פונקציות נוספות במחלקה זו. ניתן להיכנס לקוד ולצפות בהם

```
/**
 * Start run of client
 *
 * @throws IOException if can't read or write message
 */
public void runClient() throws IOException


// read the message and does things according to
// the message sent
private void proccessMsgFromServer(Message msg) throws IOException


// sends login details that client entered
// in login dialog
private void login(String errMsg) throws IOException
```

View- מחלקה המשמשת לתצוגה (GUI) של המשחק. לאחר ההתחברות לשרת, במידה והתחלנו משחק נגד AI או שהתחלנו משחק מול שחקן אנושי ושחקן כבר חיכה והמשחק התחיל מיד, התצוגה של הלוח מוצגת והמשחק מתחיל. לאחר ששחקן עשה מהלך כלשהו, הלוח הלוגי של המודל מתעדכן ובעקבותיו מתעדכן גם הלוח על ידי הפעולה האבסטרקטית updateView שנמצאת במחלקה האבסטרקטית Player.

```
+final String WIN_TITLE
+final int BUTTON_SIZE
+final Font FONT1
+final Font FONT2
+final Font BUTTON_FONT
+final int ICON_SIZE
-static int ROWS
-static int COLS
-static char EMPTY_SIGN
-ImageIcon WHITE_ICON
-ImageIcon WHITE_KING_ICON
-ImageIcon BLACK_ICON
-ImageIcon BLACK_KING_ICON
-JFrame win
-JButton btnSaveGame
-JLabel lblMsg
-JMenu StatsMenu
-JMenu optionsMenu
-JMenu userOptionsMenu
-JMenu aboutMenu
-JMenuItem gamesPlayedMenuItem
-JMenuItem gamesLastWeekMenuItem
-JMenuItem gamesVsAiMenuItem
-JMenuItem topsMenuItem
-JMenuItem deleteUserMenuItem
-JMenuItem deleteSavedGamedMenuItem
-JMenuItem ChangePwMenuItem
-JMenuItem forfeitMenuItem
-JButton[][] btnBoard
-Client client

+View(Client client)
+View(View view)
+JFrame getWin()
-void createGUI()
+void updateBoard(Board board)
+void changePassword(String errMsg)
```

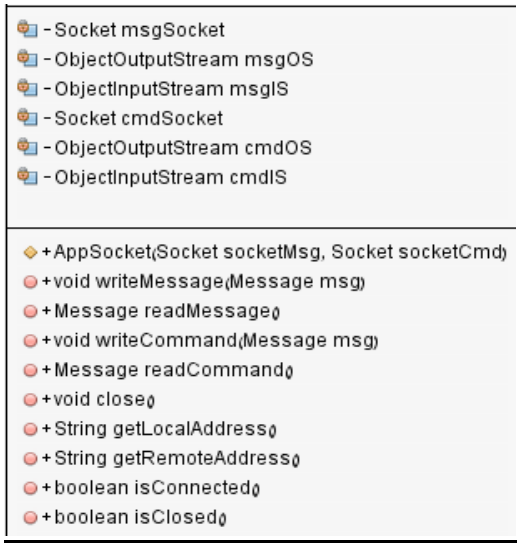
הערה: קיימות עוד פונקציות במחלקה. כדי לצפות בכל הפונקציות הקיימות יש להיכנס לקוד.

```
/**
 * Shows login window to connect to server and start play
 *
 * @param errMsg shows error message on errLabel
 * @return message with the choose of client and login
 * details (login as user, as guest or exit from login process)
 * @throws IOException if can't read or send message
 */
public Message loginDialog(String errMsg) throws IOException


/**
 * Receive board and update the btnBoard like this board
 *
 * @param board the func update btnBoard like him
 */
public void updateBoardButton(Board board)
```

4.4.5 תיעוד המחלקות המשותפות ללקוח ולשרת

1. **AppSocket-** המחלקה מייצגת את שקע התקשורת (ה- Socket) המשמש לתקשורת בין בין השרת ללקוחות. המחלקה מורכבת מ-2 שקעים : msgSocket שתפקידו לשמש עבור העברת נתונים מהשרת ללקוח וכן מהלקוח לשרת (בעיקר דברים שקשורים למשחק כמו מהלך שנעשה), ו- cmdSocket דרכו שולח הלקוח בקשות לשרת שלא קשורות למהלך המשחק כמו בקשת סטטיסטיקות.



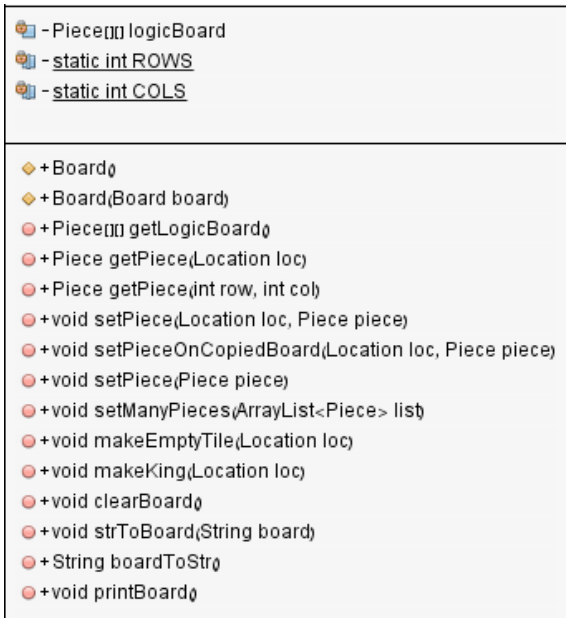

```
/**
 * Reads message by msgSocket IS
 *
 * @return the message received
 */
public Message readMessage()
```

```
/**
 * Reads messages by cmdSocket IS
 *
 * @return the message received
 */
public Message readCommand()
```

```
/**
 * Sends message by msgSocket OS
 *
 * @param msg Message that the func send
 */
public void writeMessage(Message msg)


/**
 * Sends messages by cmdSocket OS
 *
 * @param msg Message that the func send
 */
public void writeCommand(Message msg)
```

2. **Board-** מחלקה המייצגת את הלוח המשחק הלוגי. כאשר מבצעים מהלך הלוח הלוגי מתעדכן ונשלח ללקוחות ובעזרת התבנית שלו הם מעדכנים את הלוח שלהם.




```
/**
 * Constructor to create empty board
 */
public Board()
```

```
/**
 * Makes king the piece on loc
 *
 * @param loc on him the piece beccome king
 */
public void makeKing(Location loc)
```

```
/**
 * sets piece on this board on loc
 *
 * @param loc on him place the piece
 * @param piece that placed on loc
 */
public void setPiece(Location loc, Piece piece)
```

3. **Constants-** מחלקת הקבועים. משמשת בהעברת ההודעות לשני הצדדים כנושא ההודעה לפיה הצד שמקבל את ההודעה יודע כיצד להגיב.

```
+final boolean DEBUG
+final String LOGIN
+final String LOGIN_AS_GUEST
+final String LOGIN_AS_USER
+final String CANCEL_EXIT
+final String LOGIN_FAILED_MSG
+final String ALREADY_ONLINE_MSG
+final int SERVER_DEFAULT_PORT
+final String WAIT_TURN
+final String GAME_OVER
+final String GAME_OVER_TIE
+final String YOUR_TURN
+final String GET_MOVE
+final String INIT_GAME
+final String OPTIONS_FOR_MOVE
+final String UPDATE_VIEW
+final String YOU_WHITE
+final String YOU_BLACK
+final String COUNT_DOWN_FINISHED
+final String DELETE_DRAWINGS
+final String SAVE_AND_EXIT
+final String UNLOCK_SAVE_GAME_BUTTON
+final String ALL_GAMES_PLAYED
+final String TOPs
+final String YOUR_OPPONENT_EXIT
```

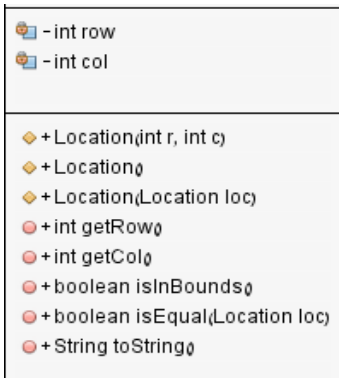
4. **GameInfo-** מחלקה האוגדת בתוכה נתונים על משחק כלשהו. משמשת עבור העברת נתונים מה-DB על סטטיסטיקות כגון : כל המשחקים ששוחקו, טופ 5 ועוד

```
- String un
- String pw
- String winner
- String gameDate
- String wins
- String generalStats

+ GameInfo(String un, String winner, String gameDate, String wins, String stats)
+ GameInfo(String un, String pw)
+ GameInfo(String generalStats)
+ String getUn()
+ String getWinner()
+ String getGameDate()
+ String getWins()
+ String getGeneralStats()
+ String getPw()
+ String toString()
```

```
/**
 * Constructor for new GameInfo
 *
 * @param un userName of player
 * @param winner the winner of game
 * @param gameDate the date that game played
 * @param wins num of wins of specific player
 * @param stats general statistics
 */
public GameInfo(String un, String winner,
        String gameDate, String wins, String stats)
```

5. **Location-** מחלקה המייצגת מיקום כלשהו בלוח. נמצאת בשימוש בעיקר כעזר עבור המחלקה Move (תפורט בהמשך) המורכבת מ- 2 לוקיישנים על הלוח.




```
/**
 * Constructor to create new location
 *
 * @param r row of location
 * @param c col of location
 */
public Location(int r, int c)
```

```
/**
 * Check if the loc is in bounds of the board
 *
 * @return true if in bounds, false else
 */
public boolean isInBounds()
```

6. **-Move** מחלקה המייצגת מהלך מסויים. מורכב מ- 2 מיקומים : מיקום המקור של הכלי ומיקום היעד.

```
-Piece piece
-Location source
-Location destination
-boolean Eat
-Location enemyLoc
-Location burnedLoc
-ArrayList<Move> secondMove

+Move(Piece player, Location source, Location dest, boolean isEat, Location enemyLoc, Location burnedLoc)
+Move(Piece player, Location source, Location destination)
+Move(Piece piece, Location source, Location dest, boolean Eat)
+Move(Piece player, Location source, Location destination, Location burnedLoc)
+Move(Piece player, Location source, Location destination, boolean isEat, Location enemyLoc)
+Move()
+Move(Move move)
+boolean hasNext()
+ArrayList<Move> getSecondMove()
+void setSecondMove(ArrayList<Move> secondMove)
+void addToSecondMove(Move secondMove)
+Piece getPiece()
+Location getSource()
+Location getDestination()
+boolean isEat()
```

הערה : קיימות עוד פונקציות. ניתן להיכנס לקוד ולצפות בהם

```
/**
 * Checks if any move include second move
 * (exists if move has double eating)
 * @return true if has secondMove, false else
 */
public boolean hasNext()
```

```java
/**
 * Constructor to create new move
 *
 * @param piece piece that make the move
 * @param source source of piece
 * @param dest destination of piece
 * @param isEat if move include eat
 * @param enemyLoc location of opponent's piece has eaten (if exists)
 * @param burnedLoc location of burned piece (if exists)
 */
public Move(Piece piece, Location source, Location dest, boolean isEat,
        Location enemyLoc, Location burnedLoc)
```

```java
/**
 * Checks if the move is in bounds of the board (legal move)
 *
 * @return true if this move in bounds of board,
 * false otherwise
 */
public boolean isInBounds()
```

7. **LoginDetails-** מחלקה המתארת נתונים של שחקן שזה עתה ביצע התחברות לשרת (שם משתמש וסיסמה אם זה משתמש רשום, ואם המשחק הוא נגד שחקן ממוחשב)

```
/**
 * Constructor to create new LoginDetails
 *
 * @param userName name of user
 * @param password password of user
 * @param withAI if player choosed to play against AI player
 */
public LoginDetails(String userName, String password, boolean withAI)
```

8. **Message-** מחלקה המייצגת הודעות העוברות בין השרת ללקוחות. ההודעה כוללת **תמיד** את נושא ההודעה (subject) ועליו מוסיפים עוד נתונים נצרכים.

```
-String subject
-Board board
-Location loc
-Move move
-ArrayList<Move> posMoves
-ArrayList<Location> locs
-LoginDetails login
-String dateBoard
-String[] changePw
-ArrayList<String> boards
-ArrayList<GameInfo> gameInfo

+Message(String subject)
+String getSubject()
+void setSubject(String subject)
+Board getBoard()
+void setBoard(Board board)
+Move getMove()
+LoginDetails getLogin()
+String getDateBoard()
+String[] getChangePw()
+void setChangePw(String[] changePw)
+ArrayList<String> getBoards()
+void setBoards(ArrayList<String> boards)
+void setDateBoard(String statsNum)
+void setLogin(LoginDetails login)
+void setMove(Move move)
+Location getLoc()
+void setLoc(Location loc)
+ArrayList<Move> getPosMoves()
+void setPosMoves(ArrayList<Move> posMoves)
+ArrayList<Location> getLocs()
+void setLocs(ArrayList<Location> locs)
+ArrayList<GameInfo> getGameInfo()
+void setGameInfo(ArrayList<GameInfo> gameInfo)
+String toString()
```

```
/**
 * Constructor to create message
 * with subject
 *
 * @param subject of message (what to do)
 */
public Message(String subject)
```

9. **-Piece** מחלקה המייצגת כלי על הלוח הלוגי. כאשר עושים מהלך כלשהו הלוח הלוגי מתעדכן בכך שהכלי נמחק מלוקיישן המקור ועובר אל היעד. במידה והכלי הגיע לקצה השני של הלוח המשתנה הבוליאני שלו king נהפך לtrue

```
- PieceColor pieceColor
- Location pieceLoc
- boolean king

+ Piece(PieceColor pieceColor, Location pieceLoc, boolean king)
+ Piece(Piece piece)
+ PieceColor getPieceColor()
+ Location getPieceLoc()
+ boolean isKing()
+ void setKing(boolean king)
+ void setPieceColor(PieceColor pieceColor)
+ void setPieceLoc(Location pieceLoc)
+ boolean isWhite()
+ char getSignOfPiece()
+ void signToPiece(char c)
+ boolean equals(Piece piece)
+ String toString()
```

```
/**
 * Constructor to create new piece
 *
 * @param pieceColor white or black piece
 * @param pieceLoc location on board
 * @param king if piece is king
 */
public Piece(PieceColor pieceColor, Location pieceLoc, boolean king)
```

```
/**
 * Checks if this piece is white player
 *
 * @return true if piece is white piece,
 * false otherwise
 */
public boolean isWhite()
```

```
/**
 * return the char the represents the piece:
 * w- ordinary white
 * W- white king
 * b- ordinary black
 * B- black king
 *
 * @return char that represents the piece
 */
public char getSignOfPiece()
```

4.5 אלגוריתמים ופעולות נבחרות

בסעיף זה נציג אלגוריתמים ופעולות נבחרות בפרויקט. על כל אחד מהם נסביר באופן מפורט מהי מטרתו, איפה השתמשנו בו בפרויקט, איך הוא פועל וסיבוכיות זמן הריצה שלו.

4.5.1 אלגוריתם מינימקס עם גיזום

כדי לממש שחקן ממוחשב, נעשה שימוש באלגוריתם מינימקס שפועל באופן הבא:
הפונקציה עוברת על כל המהלכים האפשריים של שחקן כלשהו באמצעות "עץ משחק" (אין הכוונה לעץ בינארי). עבור כל מהלך הפונקציה נותנת עליו ציון לפי הקריטריונים של פונקציית ההערכה (eval). כל ירידה ברקורסיה הפרמטר isMax משתנה מ true לfalse או להפך. כאשר המשתנה הוא true מחפשים את המהלך עם הציון הטוב ביותר (זאת אומרת, השחקן שעכשיו תורו במהלך הירידה ברקורסיה הוא השחקן שהתקבל כפרמטר) וכאשר הוא false מחפשים את המהלך עם הציון הכי פחות טוב, כלומר, מחפשים כעת את המהלך הכי פחות טוב עבור השחקן שעבורו מחפשים את המהלך החכם (תור השחקן היריב במהלך הרקורסיה). מכיוון שבדמקה ישנה אפשרות לאכילה כפולה, הפונקציה בודקת כל פעם לפני הירידה בעומק למהלך הבא האם המהלך שכעת אנו בודקים כולל אכילה כפולה. במידה והוא כולל אכילה כפולה, הפונקציה לא תשנה את isMax בירידה בעומק (מכיוון שעדיין תורו של isMax לשחק שכן הוא צריך לאכול כפול). עבור כל ירידה בעומק הפונקציה יורדת ברקורסיה ובודקת כל פעם האם עבור אותו מהלך יש ניצחון לשחקן או שהגענו לעומק שהגיע כפרמטר לפונקציה. במידה ונעצרנו בתנאי עצירה, הפונקציה מעלה את המהלך עם הציון הכי טוב למעלה ברקורסיה ומחזירה אותו.
הפונקציה מיישמת גם את אלגוריתם "גיזום אלפא בטא". המטרה של האלגוריתם היא לבצע גיזום עד כמה שניתן בענפי העץ מבלי לפגוע בתוצאה המוחזרת (כלומר, למרות הגיזומים עדיין תוחזר התוצאה הטובה ביותר). הגיזום מתחלק ל2 חלקים:

<u>גיזום אלפא:</u> אם בעומק הנוכחי אליו הגענו זהו תורו של שחקן ה"מקס" (השחקן שהתקבל כפרמטר לפונקציה)- האלגוריתם לא ימשיך לבדוק תתי עצים בעלי ציון נמוך יותר ממה שהתקבל בתת העץ הקודם. הסיבה לכך היא כיוון ששחקן ה"מקס" שואף תמיד למצוא ציון מקסימלי עד כמה שניתן. ולכן, אם נמצא מהלך מקסימלי כלשהו, אין טעם לבדוק תתי עצים עם ציון נמוך יותר.

<u>גיזום בטא:</u> אם בעומק הנוכחי אליו הגענו זהו תורו של שחקן ה"מין" (השחקן היריב של השחקן שהתקבל כפרמטר לפונקציה)- האלגוריתם לא ימשיך לבדוק תתי עצים בעלי ציון גבוה יותר ממה שהתקבל בתת העץ הקודם. הסיבה לכך היא כיוון ששחקן ה"מין" שואף תמיד למצוא ציון מינימלי עד כמה שניתן (כיוון שככל שהציון נמוך יותר הוא פחות טוב לשחקן ה"מקס" שעבורו אנו מחפשים מהלך). ולכן, אם נמצא מהלך מינימלי כלשהו, אין טעם לבדוק תתי עצים עם ציון גבוה יותר.

האלגוריתם ממומש בקוד הפרויקט ע״י הפעולה **minimax** (שנמצאת במחלקה Model.java) שנעזרת באלגוריתם להערכת מצב (פונקציית הערכה שתוסבר בסעיף הבא) שמנקד לוח משחק – עד כמה הוא טוב לשחקן.

הצגת קוד האלגוריתם

להלן קוד הפעולה:

```
private MinimaxMove minimax(PieceColor player, Board board, boolean isMax,
            int depth, int maxDepth, int alpha, int beta)
    {
        MinimaxMove bestMove = new MinimaxMove();

        if(depth == maxDepth || isGameOver(board))
        {
            bestMove.setGrade(eval(board, player));
            bestMove.setDepth(depth);
            return bestMove;
        }
        if(!isMax)
        {
            player = getOpponent(player);
        }
        ArrayList<Move> possibleMoves = getAllPossibleMoves(board, player);
        if(isMax)
        {
            bestMove.setGrade(Integer.MIN_VALUE);
            for(int i = 0; i < possibleMoves.size(); i++)
            {
                Move move = possibleMoves.get(i);
                Board newBoard = new Board(board);
                newBoard = makeMoveOnCopyBoard(newBoard, move);
                MinimaxMove minimaxMove;
                if(!move.getSecondMove().isEmpty())
                {
                    minimaxMove = minimax(player, newBoard, true, depth,
                                          maxDepth, alpha, beta);

                }
                else
                {
                    minimaxMove = minimax(player, newBoard, false, depth + 1,
                                          maxDepth, alpha, beta);
                }
                if(depth == 0)
                {
                    minimaxMove.copyMove(move);
                }
                if(minimaxMove.isDepthLegit()
                        && (minimaxMove.getGrade() > bestMove.getGrade()
                        || (minimaxMove.getGrade() == bestMove.getGrade()
                        && minimaxMove.getDepth() < bestMove.getDepth())))
                {
                    bestMove = new MinimaxMove(minimaxMove);
```

```
                }
                alpha = Math.max(bestMove.getGrade(), alpha);
                if(beta <= alpha)
                {
                    break;
                }
            }
        }
        else
        {
            bestMove.setGrade(Integer.MAX_VALUE);
            for(int i = 0; i < possibleMoves.size(); i++)
            {
                Move move = possibleMoves.get(i);
                Board newBoard = new Board(board);
                newBoard = makeMoveOnCopyBoard(newBoard, move);
                MinimaxMove minimaxMove;
                if(!move.getSecondMove().isEmpty())
                {
                    minimaxMove = minimax(player, newBoard, false, depth,
                                          maxDepth, alpha, beta);

                }
                else
                {
                    minimaxMove = minimax(player, newBoard, true, depth + 1,
                                          maxDepth, alpha, beta);

                }
                if(depth == 0)
                {
                    minimaxMove.copyMove(move);
                }
                if(minimaxMove.isDepthLegit()
                        && (minimaxMove.getGrade() < bestMove.getGrade()
                        || (minimaxMove.getGrade() == bestMove.getGrade()
                        && minimaxMove.getDepth() > bestMove.getDepth())))
                {
                    bestMove = new MinimaxMove(minimaxMove);
                }
                beta = Math.min(bestMove.getGrade(), beta);
                if(beta <= alpha)
                {
                    break;
                }
            }
        }

        return bestMove;
    }
```

המינימקס שלי עובד למשך כ- 11 שניות כאשר אופן הפעולה שלו היא DFS (צלילה לעומק). הפונקציה getBestMoveUsingMinimax שמוגדרת אצלי גורמת למינימקס לפעול איטרטיבית באופן הבא:

המינימקס מתחיל לפעול וכל פעם מגדיל ב1 את העומק אליו הוא צולל ומחזיר תוצאה, ובודק האם נשאר לו עוד זמן לסיום- במידה ויש זמן בודק את המינימקס עד העומק הבא בתור ואם לא מחזיר את המהלך האחרון שהתקבל. לדוגמה: בהתחלה המינימקס צולל לעומק 1 ומחזיר מהלך. לאחר מכן הוא בודק האם יש עוד זמן לעשות בדיקה לעומק 2. אם כן נכנס לפונקציה ובודק מהלך עבור עומק 2 ואם לא מחזיר את המהלך האחרון שהתקבל וכן הלאה...

היעילות של האלגוריתם היא O(b ^ d) כאשר b זה כמות הענפים ו- d זה העומק אליו הפונקציה מגיעה (שהתקבל כפרמטר לפונקציה). כמות הענפים היא נגזרת של הפונקציה getAllPossibleMoves (שתוסבר בהמשך), כך שככל שיהיו יותר מהלכים יהיו יותר ענפים. במשחק, למלך אפשרויות התזוזה הרבות ביותר, וכמות המהלכים במקסימלית האפשרית עבורו היא כ- 13 אם הוא עומד במשבצת מרכזית בלוח. במשחק כמות המלכים הגדולה ביותר שבדרך כלל שחקן מגיע אליה היא סביב ה- 3, כאשר הכלי על המיקום המרכזי ביותר יכול לעשות 13 מהלכים, האחד הבא אחריו יכול לעשות כ- 11 מהלכים, והבא בתור כ-9 מהלכים, ולכן כ-23 ענפים יהיו במקרה הכי גדול. בנוסף, במסגרת הזמן שהוגדר, העומק אליו הפונקציה מגיעה נע סביב 6-8.

4.5.2 פונקציית הערכה

פונקציית ההערכה (eval) תפקידה לתת ציון על לוח מסויים עבור שחקן כלשהו (שחור או לבן). הפונקציה עוזרת למינימקס לקבל החלטה האם המהלך טוב ביחס לאחרים או לא. אם עבור הלוח שהתקבל יש סיום משחק כלשהו קל לקבוע את הציון, אך אם אין סיום משחק בלוח יש צורך לקבוע בדרך כלשהי עד כמה הוא טוב לשחקן הנוכחי. הפונקציה מקבלת את הלוח אותו נבדוק ואת השחקן עבורו ניתן ציון ללוח ובודקת את הלוח. במידה ויש בלוח הנוכחי ניצחון לשחקן שהתקבל הציון שיוחזר יהיה 100, במידה ויש לו הפסד הציון יהיה 100-. במידה ובלוח שהתקבל המשחק לא נגמר, הציון יתחיל מ50 ויתעדכן במהלך הפונקציה במידת הצורך. קבעתי לפונקציה כמה פרמטרים כדי להעריך את הציון:

1. ההפרש בין כמות הכלים הרגילים (לא מלכים) שיש לשחקן הנוכחי לכמות הכלים הרגילים שיש ליריב. הציון מתעדכן כך שההפרש כפול 3 נוסף לציון ששמור כרגע. לדוגמה: במידה ולשחקן הנוכחי יש 10 כלים רגילים וליריב יש 8 כלים, ההפרש הוא 2 כפול 3 שזה 6 ולכן לציון הנוכחי של השחקן יתווספו 6 נקודות.

2. ההפרש בין כמות המלכים שיש לשחקן הנוכחי לכמות המלכים שיש ליריב. הציון מתעדכן כך שההפרש כפול 8 נוסף לציון ששמור כרגע. לדוגמה: במידה ולשחקן הנוכחי יש מלך אחד וליריב יש 2 מלכים, ההפרש הוא 1- כפול 8 שזה 8- ולכן לציון הנוכחי של השחקן יורדו 8 נקודות.

3. ההפרש בין כמות הכלים שנמצאים בשורה הראשונה אצל השחקן הנוכחי לבין אלה של היריב. הציון מתעדכן כך שההפרש כפול 2 נוסף לציון הנוכחי.

4. ההפרש בין כמות הכלים הקיימים בפינות (הימניות והשמאליות) של הלוח של השחקן הנוכחי פחות של השחקן היריב. הציון מתעדכן כך שההפרש מתווסף לציון הנוכחי.

5. ההפרש בין כמות הכלים שיכולים לזוז (לפחות תזוזה אחת) של השחקן הנוכחי פחות של היריב. הציון מתעדכן כך שההפרש מתווסף לציון הנוכחי.

6. בדיקה האם יש אפשרות ליריב לאכול כלי רגיל לשחקן הנוכחי. במידה ויש אפשרות, תורד לשחקן הנוכחי 2 נקודות מהציון.

7. בדיקה האם יש אפשרות ליריב לאכול מלך לשחקן הנוכחי. במידה ויש אפשרות, יורדו לשחקן הנוכחי 7 נקודות מהציון.

8. בדיקה האם יש אפשרות לאכילה כפולה עבור השחקן הנוכחי ועבור היריב. במידה ויש אפשרות לשחקן הנוכחי, וגם אין אפשרות שהיריב יאכל את הכלי שיכול לאכול כפול בתורו, יתווספו לשחקן הנוכחי נקודות לפי סוג הכלים שיאכלו (2 כלים רגילים, מלך ורגיל או 2 מלכים). במידה וליריב ישנה אופציית אכילה, יורדו נקודות לפי סוג הכלים שיאכלו.

להלן קוד הפונקציה:

```
public int eval(Board board, PieceColor player)
    {
        int evalScore = 50;
        PieceColor opponent = getOpponent(player);
        if(checkWin(board, player))
        {
            return 100;
        }

        if(checkWin(board, opponent))
        {
            return -100;
        }

        else
        {
            ArrayList<Move> allMoves = getAllPossibleMoves(board, player);
            ArrayList<Move> opponentMoves = getAllPossibleMoves(board,
                                                                opponent);
```

```
            int sub = subCountOrdinaryPieces(board, player);
            evalScore += (sub * 3);

            sub = subCountKings(board, player);
            evalScore += (sub * 8);

            boolean currentOneMoveKing = oneMoveKingForCurrent(player,
                                                allMoves, opponentMoves),

                    opponentOneMoveKing = oneMoveKingForOpponent(opponent,
                                                            opponentMoves);
            if(currentOneMoveKing)
            {
                evalScore += 6;
            }

            if(opponentOneMoveKing)
            {
                evalScore -= 6;
            }

            sub = subPiecesOnFirstLine(board, player);
            evalScore += (sub * 2);

            sub = subOfPiecesInSides(board, player);
            evalScore += (sub * 1);

            sub = subOfMovablePieces(player, board);
            evalScore += (sub * 1);

            boolean opponentOneMoveEaten = ordinaryCouldBeEatenNextMove(board,
                                                            opponentMoves),
            opponentKingOneMoveEaten = kingsCouldBeEatenNextMove(board,
                                                            opponentMoves);
            int currentMultEating = GradeForMultEatingCurrent(board, allMoves,
                                                            opponentMoves),
             opponentMultEating = GradeForMultEatingOpponent(board,
                                                        opponentMoves);
            if(opponentOneMoveEaten && !opponentKingOneMoveEaten &&
                                                opponentMultEating == 0)
            {
                evalScore -= 2; // ordinary could eaten
            }

            if(((opponentKingOneMoveEaten && opponentOneMoveEaten)
                    || (opponentKingOneMoveEaten && !opponentOneMoveEaten))
                    && opponentMultEating == 0)
            {
                evalScore -= 7; // king could eaten
            }

            evalScore += currentMultEating;
            evalScore -= opponentMultEating;
        }
        return evalScore;
    }
```

ניתוח יעילות: הפונקציה עוברת על המטריצה 10 פעמים, זאת אומרת 10*O(N^2) , ועוברת 5 פעמים על רשימה מקושרת (ArrayList) בגודל לא ידוע, לכן זה 5 * O(n). כיוון ש-5 ו- 10 הם זניחים ומכיוון ש- N זניח יחסית ל- N^2 הסיבוכיות הסופית תהיה: O(N^2)

4.5.3 הרצת משחק- runGame()

לאחר שבנינו בקונסטרקטור משחק חדש בין 2 שחקנים, מתחילים להריץ את המשחק בעזרת הפונקציה. תחילה, הפונקציה בודקת האם המשחק שאנו מריצים הוא משחק חדש לגמרי או משחק שנשמר במסד הנתונים והוחלט להמשיכו כעת, ולפי זה בונה את לוח המודל ומאתחלת את תור השחקן שתורו לשחק כעת. לאחר מכן, הפונקציה מתחילה את סשן המשחק: היא שולחת לשחקן שעכשיו תורו קבוע שאומר לו שעכשיו תורו וזה פותח לו את הלוח ללחיצות אפשריות כדי שיבחר מהלך, ואילו לשחקן השני שולחת קבוע שאומרת לו לחכות תור ונועלת לו את הלוח ללחיצות. כאשר השחקן שתורו לשחק לחץ על כלי כלשהו, השרת שולח לו את כל אופציות שאותו הכלי יכול לעשות והמיקומים האלו נצבעים לו על הלוח. לאחר שהשחקן בחר במשבצת חוקית כלשהי, המהלך מתבצע ומעדכן את לוח המודל של המשחק בעקבותיו מתעדכנים גם המודלים של השחקנים (במידה והמהלך שהתקבל לא חוקי, כמו אם השחקן רצה לבטל את הכלי הנבחר ולחץ עליו שוב, הפונקציה getMove מתחילה מחדש ומצפה ללחיצה על כלי אחר) ולאחר מכן, הלוח שמוצג לכל אחד מהשחקנים מתעדכן גם הוא. לאחר מכן הפונקציה בודקת האם המשחק נגמר לאחר המהלך- אם כן, מכריזה לכל אחד מהשחקנים על זהות המנצח, סופרת לאחור למשך 7 שניות ומתחילה משחק חדש. הפונקציה גם עושה בדיקה אם במשחק לפחות אחד מהשחקנים היה משתמש רשום, סטטוס המשחק גם ישמר במסד הנתונים לצורך הפקת סטטיסטיקות בהמשך. במידה והמשחק לא נגמר, הפונקציה מחליפה את תורות השחקנים והמשחק נמשך על אותו הסדר הנ"ל.

להלן קוד הפונקציה:

```
public void runGame(boolean continueGame)
    {

        new Thread(new Runnable()
        {
            @Override
            public void run()
            {
                boolean isContinue = false;
                if(continueGame)
                {
```

```
                    isContinue = true;
                }
                while(true)
                {
                    ArrayList<Move> doubleEating = new ArrayList<>();
                    if(!isContinue)
                    {
                        defineStartGame();
                    }
                    else
                    {
                        defineContinueGame();
                        isContinue = false;
                    }
                    boolean isGameOver = false;
                    while(!isGameOver)
                    {
                        //  פותח לשחקן שעכשיו תורו את המיקומים שרלוונטיים
אליו בלוח
                        currentPlayerTurn.yourTurn(); // O(N^2)
                        Move move = currentPlayerTurn.getMove(doubleEating);
                        doubleEating.clear();
                        //System.out.println("move in game: " + move);
                        if(move == null) //  אם השחקן שעכשיו תורו התנתק
יוצא מלולאת המשחק
                        {
                            gameStopped = true;
                            break;
                        }

                        model.makeMove(move);
                        p1.setModel(model);
                        p2.setModel(model);
                        p1.updateView(move);
                        p2.updateView(move);
                        model.getLogicBoard().printBoard();
                        boolean switchTurn = true;

                        // mult eating or opponent can't move
                        if(move.hasNext())
                        {
                            switchTurn = false;
                            doubleEating = move.getSecondMove();
                        }
                        isGameOver = checkGameStatus(switchTurn); // O(N^2)
                    }

                    // אם שחקן התנתק מסיים את הריצה של המשחק
                    // ושומר את הסטטוס אם אחד מהם היה שחקן רשום
                    if(gameStopped)
                    {
                        if(registerPlayerPlayed())
                        {
                            saveGamePlayerDisconnected();
                        }
                        break;
```

```
                    }

                    if(registerPlayerPlayed())
                    {
                        //לשמור במסד הנתונים את תוצאת המשחק
                        if(model.checkWin(currentPlayerTurn.getPieceColor()))
                        {
                            DB.gameOver(p1.getId(), p2.getId(),
                                    currentPlayerTurn.getId());
                        }
                        else
                        {
                            DB.gameOver(p1.getId(), p2.getId(), "TIE");
                        }
                    }
                    p1.countDownFinished();
                    p2.countDownFinished();
                }

            }

        }).start();

    }
```

ניתוח יעילות: הפונקציה החיצונית פועלת עד אשר אחד השחקנים התנתק (בצורה יזומה או לא).
הלולאה הפנימית עוברת 3 פעמים על המטריצה ב3 פעולות שונות ולכן הסיבוכיות שלה תהיה:
O(N^2) * 3 ומכיוון שה-3 זניח נישאר עם: O(N^2).

4.5.4 proccessMsgFromServer

הפונקציה מהחשובות ביותר בצד הלקוח. הפונקציה runClient רצה בלופ אינסופי כל עוד השחקן לא נסגר מסיבה כלשהי, ובכל אירטציה מחכה להודעה מהשרת. ברגע שקיבל הודעה כלשהי, הוא שולח אותה לפונקציה proccessMsgFromServer, אשר מנתחת מה צריך לעשות לפי ההודעה שהתקבלה.
לדוגמה: הלקוח קיבל הודעה שהקבוע שלה הוא: LOGIN#. ההודעה נשלחת לפונקציה שמנתחת את ההודעה ומקפיצה את חלון ה- login בעקבות כך.
להלן קוד הפונקציה:

```
private void proccessMsgFromServer(Message msg) throws IOException
    {
        String subject = msg.getSubject();

        if(subject.equals(Constants.LOGIN))
```

```
        {
            login("");
        }

        if(subject.equals(Constants.SERVER_CLOSED))
        {
            view.serverClosed();
            this.stopClient();
        }

        if(subject.equals(Constants.GAME_OVER))
        {
            GameInfo info = msg.getGameInfo().get(0);
            String gameOverMsg = info.getGeneralStats();
            view.gameOver(gameOverMsg);
            countDown(gameOverMsg);
        }

        if(subject.equals(Constants.WAIT_TURN))
        {
            view.setLabelMsg("Wait turn...");
            view.setBoardButtonsEnabled(false);
        }

        if(subject.equals(Constants.INIT_GAME))
        {
            msg.getBoard().printBoard();
            view.setup(msg.getBoard());
            view.initGame();
        }

        if(subject.equals(Constants.UNLOCK_SAVE_GAME_BUTTON))
        {
            view.unlockSaveGameButton();
        }

        if(subject.equals(Constants.YOU_WHITE))
        {
            String title = view.getWin().getTitle();
            String newTitle = title + " (white)";
            view.getWin().setTitle(newTitle);
        }

        if(subject.equals(Constants.YOU_BLACK))
        {
            String title = view.getWin().getTitle();
            String newTitle = title + " (black)";
            view.getWin().setTitle(newTitle);
        }

        if(subject.equals(Constants.YOUR_TURN))
        {
            view.setLabelMsg("Play turn!");
            view.setBoardButtonsEnabled(false);
            ArrayList<Location> locsToUnlock = msg.getLocs();
            view.setEnabledList(locsToUnlock, true);
```

```
        }

        if(subject.equals(Constants.OPTIONS_FOR_MOVE))
        {
            ArrayList<Move> posMoves = msg.getPosMoves();
            view.paintAllpossibleMoves(posMoves);
        }

        if(subject.equals(Constants.UPDATE_VIEW))
        {
            view.cleanAllBorders();
            msg.getBoard().printBoard();
            Board board = msg.getBoard();
            view.updateBoard(board);
            view.deleteAllPaints();
            view.makeBorders(msg.getMove());
        }

        if(subject.equals(Constants.DELETE_DRAWINGS))
        {
            view.deleteAllPaints();
        }

        if(subject.equals(Constants.YOUR_OPPONENT))
        {
            String title = view.getWin().getTitle();
            view.getWin().setTitle(title + msg.getLogin().getUserName());
        }

        if(subject.equals(Constants.ALL_GAMES_PLAYED))
        {
            view.printAllGamesPlayed(msg);
        }

        if(subject.equals(Constants.TOP5))
        {
            view.printTop5(msg);
        }

        if(subject.equals(Constants.GAMES_PLAYED_LAST_WEEK))
        {
            view.printGamesLastWeek(msg);
        }

        if(subject.equals(Constants.GAMES_PLAYED_VS_AI))
        {
            view.printGamesVsAI(msg);
        }

        if(subject.equals(Constants.YOUR_OPPONENT_EXIT))
        {
            view.yourOpponentExit();
        }

        if(subject.equals(Constants.CHANGE_PASSWORD))
        {
```

```
            view.passwordChanged();
        }

        if(subject.equals(Constants.WRONG_PASSWORD))
        {
            view.changePassword("Wrong password. please enter your password
                                                                    again");
        }

        if(subject.equals(Constants.DELETE_SAVED_GAMES))
        {
            view.deleteSavedGamesSuccess();
        }

        if(subject.equals(Constants.DELETE_SAVED_GAMES_FAILED))
        {
            view.deleteSavedGamesFailed();
        }

        if(subject.equals(Constants.SELECT_UNFINISHED_BOARD))
        {
            ArrayList<String> allBoards = msg.getBoards();
            selectUnfinishedBoard(allBoards);
        }

        if(subject.equals(Constants.UNLOCK_REGISTER_OPTIONS))
        {
            view.setEnebledRegisterOptions(true);
        }

    }
```

ניתוח יעילות: הפונקציה מכילה הרבה אפשרויות פעולה והיא פועלת בהתאם לנושא ההודעה ועושה פונקציה אחת מתוך הרבה אופציות. מכיוון שהסיבוכיות הכי גדולה מכל הפונקציות יכולה להיות O(N^2) זאת תהיה הסיבוכיות הסופית.

4.5.5 getAllPossibleMoves

פונקציה חשובה מאוד בצד השרת. הפונקציה מקבלת את צבע השחקן (שחור או לבן) ומחזירה את כל המהלכים האפשריים שהשחקן יכול לבצע עם הכלים השונים שלו. הפונקציה נמצאת במחלקה Model ונמצאת בשימוש רב בצד השרת, לדברים כמו חיפוש אחרי שרופים (יש צורך לעבור על כל המהלכים האפשריים ולראות אם יש מהלך אכילה אחד לפחות וקיימים מהלכים אחרים שלא כוללים אכילה, ואז מתקיים עניין השרופים) ובמינימקס.

להלן קוד הפונקציה:

```
public ArrayList<Move> getAllPossibleMoves(Board board, PieceColor playerSign)
    {
        ArrayList<Move> allMoves = new ArrayList<>();
        for(int row = 0; row < ROWS; row++)
        {
            for(int col = 0; col < COLS; col++)
            {
                Location loc = new Location(row, col);
                Piece p = board.getPiece(loc);
                if(p != null)
                {
                    if(p.getPieceColor() == playerSign)
                    {
                        ArrayList<Move> posMoves =
                         possibleMovesForPiece(board, loc, false);
                        allMoves.addAll(posMoves);
                        posMoves.clear();
                    }
                }
            }
        }
        ArrayList<Move> updatedMoves = setBurnedLocForAllMoves(allMoves,
                                                                playerSign);
        return updatedMoves;
    }
```

ניתוח יעילות: הפונקציה עוברת על מטריצה בגודל nXn ולכן הסיבוכיות תהיה O(n^2).

4.6 מבני נתונים עיקריים

מערך חד ממדי- השתמשתי במערכים בקוד בין היתר ביצירה של מערך אובייקטים, שהוכנס לחלון של שינוי ססמה. היה בזה צורך כיוון שכדי ליצור את החלון עם כמה אובייקטים נדרש דווקא מערך ולא מבנה נתונים אחר.

Object[]

המערך כולל בתוכו אובייקטים שקשורים לחלון (תוויות ותיבות טקסטים) ואותו לאחר מכן הכנסנו לחלון

מטריצה- השתמשתי במטריצה כדי ליצור את הלוח הלוגי של המשחק. היה בזה צורך כי הייתי צריך ליצור לוח לוגי באותו גודל כמו לוח המשחק הרגיל של 8X8 ורק מטריצה יכלה למלא דרישה זו.

Piece[][]

ניתן לראות שהמטריצה היא מסוג Piece, זאת אומרת יש עליה כלים של שני השחקנים וכאשר נעשה מהלך מזיזים את הכלים שעליה על גבי המטריצה והיא מתעדכנת בהתאם.

רשימה מקושרת (ArrayList)- השתמשתי הרבה ברשימה מקושרת בקוד. אחד השימושים היה חיפוש כל התזוזות האפשרויות ששחקן יכול לבצע עם כלי מסויים. היה צורך דווקא ברשימה מקושרת כיוון שאי אפשר לצפות מראש כמה מהלכים אותו כלי יוכל לבצע היה צורך דווקא במבנה נתונים דינאמי כמו רשימה.

ArrayList<Move>

כמו שניתן לראות, הרשימה היא מסוג Move, כלומר אוגדת בתוכה מהלכים אפשריים עבור שחקן מסויים.

5. סיכום אישי

בסעיף זה אפרט סיכום אישי על תהליך הלמידה בשנתיים האחרונות: עם איזה אתגרים וקשיים התמודדתי, מה הפרויקט תרם לי, הצעות לשיפור הפרויקט שלא הספקתי לעשות ומסקנות שהסקתי.

5.1 אתגרים וקשיים איתם התמודדתי

במהלך כתיבת קוד הפרויקט התמודדתי עם קשיים רבים. הקושי המרכזי שלי היה כתיבת המינימקס וההבנה איך עובדת רקורסיה. לקח לי הרבה זמן להבין הכל איך זה עובד הרבה בגלל שהלימודים בשנה שעברה דרך הזום הקשו מאוד על ההבנה שלי בחומר, ודרשו ממני גם להשלים את החוסרים תוך כדי תנועה השנה. קושי נוסף שהיה לי היה קשור בבניית המודל המורכב שכלל הרבה חוקיות של המשחק והמון כתיבת פונקציות (במיוחד לקח לי הרבה זמן לעשות את ה"שרופים"). בנוסף, עוד קושי משמעותי שהיה לי היה קשור בניפוי ומציאת באגים. לפעמים לקח לי הרבה זמן עד שהייתי מבין את הבעיה ודרכה למדתי איך לכתוב נכון יותר קוד כדי להימנע משגיאות עתידיות. עם כל הבעיות האלו התמודדתי באמצעות הרבה עזרה מחברים והמורה אבל בעיקר באמצעות "שבירת ראש"- ניסיון למצוא לבד את הבעיה ולהתמודד בכוחות עצמי מכיוון שאני מאמין שכאשר אני מתמודד לבד אני גם מבין יותר טוב מה כתבתי וגם כתיבת הקוד שלי נעשית יותר נכונה עם פחות שגיאות.

5.2 מה הפרויקט תרם לי

הפרויקט תרם לי המון ידע וכלים במהלך כתיבתו. ראשית, הוא תרם לי בתכנון הפרויקט וניהול זמן. העובדה שלמדנו וכל חלק זמן עבדנו על חלק אחר בפרויקט ובכך הוא נפרס על השנה כולה. זה לימד אותי לא רק בפרויקט אלא גם לחיים עצמם, מכיוון שיש לי בעיה של ניצול זמן (הרבה פעמים השנה הייתי מגיע יום לפני דדליין של הגשת מטלה או מבחן בלחץ להספיק הרבה דברים שלא עשיתי) והדבר לימד אותי איך לנצל את הזמן יותר טוב לכל תחום בחיים. שנית, התחדדו לי הרבה דברים במהלך הכתיבה- למדתי בתיכון מגמת תוכנה והיו דברים שלא הבנתי עד הסוף כמו נושאים מסויימים במחלקות שכיום אני שולט בהם בצורה הרבה יותר טובה ומבין אותם הרבה יותר טוב. בנוסף לכך, קיבלתי כלים הקשורים בבניית תרשימים כמו UML ו– sequence diagrams וניתוחם. למדתי גם איך לקרוא אלגוריתמים ולכתוב אותם בהתאם לקוד שלי ואיך שהוא בנוי ולעשות בו את השינויים הנדרשים, כמו אלגוריתם המינימקס שהתאמתי אל הפרויקט שלי.

5.3 הצעות לשיפור

רציתי מאוד להוסיף לפרויקט אפקטים ומוזיקה שיהפכו אותו לחווייתי יותר, לדוגמה: כאשר כלי כלשהו נהפך למלך לעשות מוזיקה של מחיאת כפיים. כמו כן, מאוד רציתי לשנות את החלונות לתמונות יותר יפות שיוסיפו צבע, לדוגמה: חלון ה- login להשים שם תמונה יותר מעניינת וצבעונית כמו מנעול ומפתח או להקפיץ חלון כשנגמר סשן משחק בודד עם שם המנצח ומדליה עם הספרה 1 ועליו הספירה

לאחור או בחלון של טופ 5 לעשות תמונה של גביע וליד שמות השחקנים במקומות 1-3 לעשות מדליות זהב, כסף וארד.

5.4 מסקנות

למדתי הרבה דברים במהלך השנתיים האחרונות במהלך כתיבת הפרויקט שלאו דווקא קשורים גם כן בכתיבת קוד. במבט לאחור למדתי שלא כדאי לדחות דברים ומטלות אלא לפעול כמה שיותר מהר לסיים אותם ואם הייתי חוזר אחורה הייתי פועל כדי לעשות את המטלות מהר ולא להתעכב איתם לימים האחרונים שלפני ההגשה, שאז יושבים בלחץ ולא תמיד מספיקים את המטלה במלואה. דבר נוסף שהסקתי זה שלמרות הקושי שחוויתי בלימודים אני מאוד נהנה מלימודי תוכנה מאז התיכון ושאני רוצה בעתיד להתקדם ואפילו להשלים למהנדס. החוויה של הלימודים באופן כללי הייתה מאוד טובה ומעניינת, בגלל שזו צורת למידה אחרת שנכנסת לראש בצורה יותר טובה ומכניסה הרבה נושאים יחד בצורה פרקטית ולא תאורטית. אלו היו שנתיים מהנות ומאתגרות כאחת ואני לוקח איתי להמשך המון ידע שצברתי במהלך השנתיים האלו, את החברים איתם למדתי והרבה חוויות שצברתי.

6. תודות

ברצוני להודות לכל מי שתמך ועזר בכתיבת הפרויקט:
למורה אילן פרץ על כל הלמידה לאורך השנתיים האחרונות שהעשירה את הידע שלי ועל העזרה לאורך התקופה בכתיבת הפרויקט, להורים שתמכו, לחברים שעודדו ועזרו כשהייתי צריך עזרה בין בפרויקט ובין בכלל, וכמובן התודה הכי גדולה לבורא עולם שסייע בידי לעשות את הפרויקט הזה ובלעדיו לא הייתי יכול לכתוב אפילו שורת קוד אחת.

7. ביבליוגרפיה

- נעזרתי בוויקיפדיה כדי ללמוד יותר לעומק את החוקיות של המשחק:

https://he.wikipedia.org/wiki/דמקה

- נעזרתי באתר W3Shcools כדי למצוא דברים שקשורים ב-SQL:

https://www.w3schools.com/sql

- נעזרתי באתר Stack OverFlow כדי ללמוד דברים שלא ידעתי לפני (כמו מסגרות למשבצת בלוח- borders) בכתיבת קוד:

https://stackoverflow.com/questions/5716390/how-to-set-border-on-jpanel

- בקובץ PDF הזה נעזרתי כדי למצוא רעיונות לפונקציית ההערכה שלי:

https://pages.mini.pw.edu.pl/~mandziukj/PRACE/es_init.pdf

- נעזרתי בוויקיפדיה על מודל שרת לקוח כדי להעמיק את הידע שלי על הנושא:

https://he.wikipedia.org/wiki/שרת–לקוח